

©Copyright 2024

Gautam Kumar

Foodereum: Revolutionizing Food Supply Chains with Blockchain Authentication

Gautam Kumar

A Master's Project

submitted in partial fulfillment of the
requirements for the degree of

Master of Science in Cybersecurity Engineering

University of Washington

2024

Reading Committee:

Dr. Brent Lagesse, Chair

Dr. Marc Dupuis

Dr. Ahmed Awad

Program Authorized to Offer Degree:

Master of Science in Cybersecurity Engineering

University of Washington

Abstract

Foodereum: Revolutionizing Food Supply Chains with Blockchain Authentication

Gautam Kumar

Chair of the Supervisory Committee:
Associate Professor Dr. Brent Lagesse
Division of Computing & Software Systems

Blockchain technology has the potential to revolutionize supply chain management by enhancing transparency, traceability, and security. The proposed work suggests adopting a private Ethereum-based blockchain system to record, track, and verify food products throughout the supply chain. The immutability of blockchain ensures that transactions cannot be altered or concealed, as they are recorded in a distributed ledger accessible to all authorized parties. This transparency allows receivers to access the entire history of a product on the blockchain network, improving food quality and safety. By implementing this blockchain-based solution, the food industry aims to increase consumer appeal and overcome challenges such as centralized systems, third-party involvement, high costs, food waste due to spoilage and contamination, lack of accountability, and communication gaps between supply chain partners.

However, the system faces security threats like cross-site scripting (XSS) attacks, sensitive information exposure, and hardcoded credentials. To mitigate these risks, the project incorporates robust input validation, output encoding, secure storage, and strong authentication mechanisms, including two-factor authentication and strong password policies. Regular security audits, penetration testing, and software updates are also implemented to identify and address vulnerabilities. These security enhancements, combined with blockchain's inherent benefits, create a more robust and trustworthy system for food supply chain management,

increasing transparency, and efficiency, and providing a significantly more secure environment for all stakeholders involved in the food supply chain by increasing consumer appeal and overcoming challenges such as centralized systems, third-party involvement, high costs, food waste, lack of accountability, and communication gaps between supply chain partners.

TABLE OF CONTENTS

	Page
List of Figures	iii
Glossary	iv
Chapter 1: Introduction	1
1.1 Food Supply Chain	1
1.2 Blockchain Technology	1
1.3 Authenticated solution for Food Supply Chain	4
1.4 Organisation of the Project Report	6
Chapter 2: Problem Statement and Proposed Solution	7
2.1 Problem Statement	7
2.2 Proposed Solution	8
2.3 Scope and Requirements	9
Chapter 3: Related Work	14
3.1 Existing Systems	14
3.2 Background	18
Chapter 4: Methodology	23
4.1 Selection of Methodologies and Tools:	23
4.2 OWASP Guidelines and Best Practices:	24
4.3 Additional Techniques Considered:	25
4.4 Tool Selection Rationale:	26
4.5 Static Analysis Enhancements:	27
4.6 Dynamic Analysis Enhancements:	27
4.7 Demonstration of Testing and Findings:	28

4.8	Penetration Testing	29
4.9	Challenges and Unsuccessful Approaches:	30
Chapter 5:	Implementation	31
5.1	Implementation Platforms	31
5.2	Implementation Details	31
Chapter 6:	Results	36
6.1	Static Analysis Results:	36
6.2	Dynamic Analysis Results:	36
6.3	Summary of Enhancements:	37
6.4	Areas of Inconclusive Testing:	37
6.5	Analysis of Gas Consumption by Smart Contract & its Functions	44
Chapter 7:	Conclusion and Future Enhancements	46
7.1	Conclusion	46
7.2	Future Enhancements	47
Bibliography		51
Appendix A:	Solidity Smart Contract Code	54
Appendix B:	Screenshots	57

LIST OF FIGURES

Figure Number	Page
3.1 Layered architecture of AgriBlockIoT	14
3.2 Proposed Framework of Monitoring Crop Price	15
3.3 Reputation System	16
3.4 System Model	19
5.1 Ganache Transactions	32
5.2 Metamask with Personalized Network	33
5.3 Remix IDE	34
5.4 SYNK Penetration Testing	35
6.1 XSS on login.php page	38
6.2 XSS on manage.php page	39
6.3 Information Exposure on login.php page	40
6.4 Use of Password Hash on login.php page	41
6.5 Use of Hardcoded Credentials on connectdb.php page	42
6.6 Use of Hardcoded Credentials on config/Database.php page	43
6.7 Gas Consumption vs Smart Contract and its Functions	44
B.1 Sanction Products Page	57
B.2 Scan & Update Products Page	58
B.3 Track Products Page	59
B.4 List of Notifications	60

GLOSSARY

ACCOUNTABILITY: The responsibility and answer-ability for actions taken, ensuring that individuals or entities are held liable for their performance or behavior.

AUTHENTICATION: The process of verifying and validating the identity and integrity of a product, person, or entity.

BLOCKCHAIN TECHNOLOGY: A distributed and decentralized digital ledger that records transactions in a secure and immutable manner.

DECENTRALISED APPLICATION - DAPP: An application that operates autonomously through smart contracts on a decentralized computing system.

ETHER: The native cryptocurrency of the Ethereum blockchain platform.

ETHEREUM: A decentralized global software platform powered by blockchain technology, known for its native cryptocurrency, Ether (ETH).

GANACHE: A private Ethereum blockchain environment that allows users to emulate the Ethereum blockchain for interacting with smart contracts in a private setting.

META-MASK: A popular self-custodial wallet that allows users to interact with the Ethereum blockchain through a browser extension or mobile app, enabling them to store, manage, and exchange Ethereum-based cryptocurrencies and tokens securely.

MYSQL: An open-source relational database management system (RDBMS) for organizing data in one or more data tables with relations.

OWASP: The Open Web Application Security Project (OWASP) is a nonprofit foundation that provides guidance on how to develop, purchase and maintain trustworthy and secure software applications. OWASP is noted for its popular Top 10 list of web application security vulnerabilities.

PENETRATION TESTING: The practice of testing a computer system, network, or web application to find security vulnerabilities that could be exploited by malicious actors.

SECURITY: The state of being protected from threats, dangers, or loss.

SMART CONTRACTS: Self-executing contracts with the terms of the agreement directly written into code, automatically enforcing the agreed-upon conditions.

SUPPLY CHAIN MANAGEMENT: The coordination and management of the flow of goods, services, and information across multiple entities involved in the production, distribution, and delivery of products.

TRACEABILITY: The ability to track and verify the history, location, and attributes of a product or item throughout its lifecycle.

TRANSPARENCY: The visibility and openness of information, ensuring that data and processes are accessible and understandable to relevant stakeholders.

REMIX IDE: A powerful toolset for developing, deploying, debugging, and testing Ethereum and EVM-compatible smart contracts.

Chapter 1

INTRODUCTION

1.1 Food Supply Chain

The food industry plays a vital role in supplying essential goods and meeting the needs of numerous human activities and behaviors. Once food is harvested or manufactured, it must be preserved, distributed, and retailed so that it reaches end consumers on time. However, reports indicate that over one-third of all food produced is abandoned or wasted each year, with two-thirds of this waste occurring in the supply chain, including during harvesting, transportation, and warehousing. For example, megatons of fruits and vegetables are wasted worldwide due to inefficient and inadequate food supply chain management (FSCM) [1], [2]. As a result, effective food supply chain management is critical in reducing food waste and ensuring food security. The global economy relies heavily on supply chains, and food supply chains are particularly complex systems with multiple actors involved [3]. Currently, conventional records in food supply chains primarily monitor and store orders and shipments, without incorporating essential features such as transparency, traceability, and auditability. Furthermore, communication gaps between supply chain partners, the possibility of food fraud, rising supply chain costs, and growing regulations highlight the system's inefficiencies [4], [5]. Addressing these challenges can improve food quality and safety, which is becoming increasingly important among consumers. Tackling these hurdles will require innovative solutions and a collaborative approach among all stakeholders in the food supply chain.

1.2 Blockchain Technology

Blockchain technology is a decentralized, immutable, and distributed ledger that can replace traditional centralized databases for recording transactions. In a blockchain, transactions

are validated by network nodes and recorded in a new block, ensuring trust and reliability across the supply chain. The stored assets are tamper-proof because all entries in the blockchain are based on a consensus reached by the majority of the network's peers, making the ledger immutable by design and providing an auditable and transparent source of information. Blockchain technology has been widely adopted in various industries, such as banking, healthcare, IoT, insurance, and supply chain management, and has proven to be highly effective in combating fraud and theft [6].

1.2.1 Application of Blockchain Technology in Food Supply Chain

Blockchain technology has emerged as a promising solution for improving the efficiency and transparency of food supply chains. The use of blockchain in the food supply chain has gained significant traction in recent years, with several prominent use cases [7], [8], [9], [10], [11].

One of the key benefits of blockchain in the food supply chain is improved traceability. Blockchain allows organizations to track the origin, movement, and transformation of food products throughout the supply chain [7], [8], [9], [10], [11]. This enhanced traceability can help prevent food fraud and ensure food safety by enabling rapid identification of the source of contamination or quality issues [7], [10], [11].

Blockchain also enhances transparency in the food supply chain by providing a decentralized, immutable record of all transactions and events [7], [8], [9], [10], [11]. This transparency can help build trust among consumers and promote sustainable and ethical sourcing practices [7], [8], [9].

Furthermore, blockchain-based smart contracts can automate various supply chain processes, such as payments, quality checks, and compliance verification [8], [9], [10], [11]. This can lead to increased efficiency, reduced costs, and better coordination among supply chain participants [8], [9], [10], [11].

In the agricultural sector, blockchain can enable farmers to participate in the supply chain without the involvement of intermediaries, allowing for more direct transactions and better traceability [7], [8], [9], [11]. The combination of blockchain with other digital technologies,

such as QR codes, RFID, and sensors, can further enhance the digitization and automation of the food supply chain [8], [9], [11].

Overall, the use of blockchain in the food supply chain has the potential to improve traceability, enhance transparency, and increase efficiency, making it a valuable tool for addressing the challenges faced by the industry [7], [8], [9], [10], [11].

1.2.2 Decentralised Application - DAPP

Decentralized applications (dapps) are applications that combine a smart contract and a frontend user interface and are built on a decentralized network [12].

1.2.3 Metamask

MetaMask is a popular Ethereum wallet that allows users to interact with decentralized applications (dApps) by providing access to their Ethereum wallet through a browser extension or mobile app [13], [14].

1.2.4 Ganache

Ganache is a software tool that developers widely use to create a local blockchain network for testing and development purposes [15].

1.2.5 Remix IDE

Remix IDE is a popular programming environment for developing Ethereum smart contracts. It allows users to rapidly compile and deploy Solidity programs on local virtual machines or external Web3 service providers like MetaMask [16]. Solidity is a high-level, object-oriented language used to create smart contracts that control how accounts behave in the Ethereum state [16].

1.3 Authenticated solution for Food Supply Chain

In this project, we present Foodereum, an Ethereum-based authentication system for food programs that allows organizations to record, track, and authenticate food products along the supply chain. Since immutability is a virtue of the blockchain, transactions cannot be tampered with or hidden because every transaction is recorded in a distributed ledger, which also can be traced and revealed to the whole network who have access to the data. This implies that anytime a product is delivered, a new receiver will be able to view the product's entire history on the blockchain network. A comprehensive performance analysis of the proposed method in terms of gas consumption has been reviewed. Blockchain overcomes the difficulties by offering a balance in a sophisticated supply chain. Since there are no third parties engaged in transactions, and authorisation and everything is accomplished by consensus, both users and system operators must follow a set of rules to keep the system working. In this proposed model, the food products have to be authenticated upon shipment at each level or region. The authentication in the system establishes a tamper-proof environment in the supply chain, which eliminates fraudulent activities, ensures accountability of the entities involved, maintains data integrity, and also provides complete transparency in the system. As consumers seek assurance for the food they consume, an authentication system in the food supply chain, allows consumers to be treated with transparency and openness which enables them to recognize and consume high-quality food. The components of the supply chain are:

- **Sanctioner / Manufacturer:** Who is responsible for sanctioning or producing the food commodities.
- **Distributors:** These entities are in charge of authenticating the shipments and transporting the food commodities from place to place that are sanctioned by the Sanctioner / Manufacturer.
- **Consumers:** The end users who receive the food products.

Each transaction such as the process of registering or sanctioning the food products in the network and authenticating it for shipment will be recorded as blocks, which enables to achieve of complete traceability in the system and maintains trust among the consumers in the food supply chain.

1.3.1 Reason for the new approach to work

The proposed approach utilizing an Ethereum-based blockchain network is expected to work for several reasons:

1. **Transparency:** Blockchain technology provides a decentralized and immutable ledger, ensuring transparency in the food supply chain. Every transaction, verification, and update made on the blockchain is recorded and visible to all participants, eliminating the possibility of tampering or manipulation of data. This transparency fosters trust among stakeholders and allows for real-time monitoring of the distribution process.
2. **Improved Quality Assurance:** By utilizing QR codes or Product IDs authenticated by government officials at each level of the supply chain, the proposed system enables accurate tracking and verification of food products. This ensures that only quality products that meet the specified standards are distributed. Any inconsistencies or deviations can be identified quickly, enabling timely corrective actions to maintain the quality of food commodities.
3. **Prevention of Wastage:** With a transparent and traceable supply chain, the proposed system can help prevent food wastage. By tracking the movement of food products from production to distribution, inefficiencies, and bottlenecks can be identified, enabling better planning and optimization of the supply chain. This reduces the likelihood of food products expiring or becoming unfit for consumption before reaching the beneficiaries.

4. **Accountability and Elimination of Corruption:** The use of smart contracts on the blockchain enhances accountability in the distribution process. Smart contracts can automate and enforce predefined rules and regulations, ensuring that the government administration adheres to the specified protocols. This eliminates the potential for corruption or favoritism in the distribution of food commodities, as the system operates based on predefined and transparent rules.
5. **Real-time Tracking and Monitoring:** The proposed system allows for real-time tracking and monitoring of food products. Using QR codes or Product IDs, any entity in the network can scan and track the status and location of food commodities. This not only enables efficient inventory management but also enhances the ability to respond promptly to any issues or emergencies that may arise during the distribution process.

1.4 Organisation of the Project Report

The project report is organized as given below: In Chapter 2, we discuss the problem statement, the solution to the problem, and its scope and requirements. The Chapter that follows i.e. Chapter 3 consists of the details on the related works or existing systems to the problem statement and the background of the project. In Chapter 4, we present the Methodology which includes the different security analysis followed by the OWASP guidelines. The next chapter, Chapter 5 gives the requirements and details about the implementation of the proposed system along with the platforms used in the project. In Chapter 6, we discuss the result of the project. Chapter 7 concludes the paper along with a mention of the Future Enhancements. The other screenshots and the source code are gathered in the Appendix.

Chapter 2

PROBLEM STATEMENT AND PROPOSED SOLUTION

2.1 Problem Statement

The problem is challenging due to the food supply chain's complexity and multiple stakeholders' involvement. Coordinating and monitoring the various actors in the supply chain, such as farmers, suppliers, distributors, and government agencies, requires significant effort and coordination. Furthermore, ensuring the integrity and accuracy of data throughout the supply chain, particularly in a decentralized and transparent manner, can be a technically demanding task. Overcoming issues related to data privacy, scalability, and interoperability of blockchain-based solutions adds further complexity to the problem.

To tackle this, we built an authenticated food supply chain system that provides efficient traceability, and transparency of information, improves the quality and prevents wastage of food, and ensures accountability of Supply Chain entities involved in distributing food commodities.

However, the existing food supply chain authentication system built suffers from several critical security vulnerabilities that undermine its integrity and reliability. These issues have been identified through extensive static and dynamic analysis.

Firstly, the system is vulnerable to Cross-Site Scripting (XSS) attacks on both the login and management pages. This allows malicious actors to inject harmful scripts that can compromise user data and system operations. Secondly, there is a significant risk of sensitive information exposure due to insecure error handling and logging practices. This flaw can lead to unauthorized access to confidential data, further endangering the system's security.

Additionally, the use of weak password hashing mechanisms is a major concern. Weak hashing increases the risk of credential compromise, making it easier for attackers to gain

unauthorized access to the system. Furthermore, the presence of hardcoded credentials in database connection files poses a severe security threat. Hardcoded credentials can be easily exploited, providing attackers with direct access to the database and sensitive information stored within.

These vulnerabilities expose the system to various attacks, including unauthorized access, data breaches, and manipulation of supply chain information. The lack of comprehensive security testing and auditing processes exacerbates these issues, leaving potential weaknesses undetected and unaddressed.

2.2 *Proposed Solution*

To mitigate these risks, the main focus of this capstone project is to address critical security vulnerabilities in the existing food supply chain authentication system through a comprehensive set of improvements. To combat injection attacks and cross-site scripting vulnerabilities, particularly on login and management pages, the solution enhances input validation and output encoding mechanisms. Secure storage and handling of sensitive information are implemented to prevent exposure through insecure error handling and logging practices. The system replaces weak password hashing mechanisms with stronger alternatives and introduces robust authentication methods, potentially including multi-factor authentication, to mitigate the risk of credential compromise.

A significant security enhancement involves the elimination of hardcoded credentials, especially in database connection files, and replacing them with secure credential management practices. To maintain ongoing security, the solution incorporates regular security audits and penetration testing into the development and maintenance processes. This proactive approach aims to identify and address potential vulnerabilities before they can be exploited. Additionally, the implementation of secure coding practices throughout the development lifecycle, including code review processes and security checks integrated into the CI/CD pipeline, further strengthens the system's overall security posture.

By implementing these security enhancements, the proposed solution aims to create a

more robust and trustworthy food supply chain authentication system. These measures significantly reduce the risk of unauthorized access, data breaches, and manipulation of supply chain information. The result is a system that ensures the integrity of product tracking and verification processes throughout the supply chain, providing stakeholders with a secure and reliable platform for managing food supply chain operations.

2.3 Scope and Requirements

2.3.1 Scope

The scope of the Foodereum project encompasses enhancing the security of an existing Ethereum-based blockchain authentication system for food supply chain management. The project aims to address critical security vulnerabilities identified in the existing system and implement robust security measures to ensure the integrity and reliability of the system. This includes enhancing the authentication process for food products throughout the supply chain, from sanctioning/manufacturing to distribution and consumption, as well as improving traceability and transparency of food products by recording all transactions on the blockchain. The scope covers both static and dynamic analysis aspects of the system, with specific enhancements in input validation, output encoding, secure storage of sensitive information, elimination of hardcoded credentials, integration of strong authentication mechanisms (including two-factor authentication), secure session management, and regular security audits and penetration testing. Additional security measures within the scope include implementing rate limiting and account lockout mechanisms, securely configuring the web server and database, and establishing a regular software update and patch management process. The project also includes demonstrating the testing and findings of these security enhancements, as well as analyzing the gas consumption of the smart contract and its functions. By implementing these security enhancements, the project aims to create a more robust and trustworthy system for food supply chain management, complementing the inherent benefits of blockchain technology such as transparency and immutability.

2.3.2 Requirements

The key requirements and security changes include:

1. Static Analysis Enhancements:

- **Input Validation and Output Encoding:** Implement robust input validation and output encoding to prevent Cross-Site Scripting (XSS) attacks. This ensures that any data entered by users is properly sanitized before being processed or displayed.
- **Secure Storage of Sensitive Information:** Ensure that sensitive information, such as passwords and confidential data, is stored securely using strong encryption techniques. This prevents unauthorized access and data breaches.
- **Elimination of Hardcoded Credentials:** Remove any hardcoded credentials from the codebase to prevent unauthorized access. Credentials should be securely managed and stored using environment variables or secure vaults.

2. Dynamic Analysis Enhancements:

- **Strong Authentication Mechanisms:** Integrate strong authentication mechanisms, including two-factor authentication (2FA) and enforcement of strong password policies. This adds an additional layer of security to user accounts.
- **Secure Session Management:** Implement secure session management practices to prevent session hijacking and unauthorized access. This includes using secure cookies, setting appropriate session timeouts, and ensuring sessions are properly invalidated upon logout.
- **Regular Security Audits and Penetration Testing:** Conduct regular security audits and penetration testing to identify and address potential vulnerabilities. This proactive approach helps in maintaining the security posture of the system.

3. Additional Security Measures:

- **Rate Limiting and Account Lockout Mechanisms:** Implement rate limiting and account lockout mechanisms to prevent brute force attacks. This ensures that repeated failed login attempts are detected and mitigated.
- **Secure Configuration of Web Server and Database:** Ensure that the web server and database are securely configured to minimize attack surfaces. This includes disabling unnecessary services, applying security patches, and using secure communication protocols.
- **Software Updates and Patch Management:** Regularly update and patch the software to address known vulnerabilities. Keeping the system up-to-date with the latest security patches is crucial for maintaining its security.

4. Security and Privacy:

- The system must prioritize the security and privacy of sensitive data stored on the blockchain.
- Encryption techniques, secure authentication mechanisms, and regular security audits should be implemented to protect against unauthorized access, data breaches, and tampering.
- MetaMask, a browser plugin, acts as a gateway between the Ganache blockchain network and the Remix IDE, ensuring secure communication.

5. Scalability and Performance:

- The system should be designed to oversee a large volume of transactions and data to accommodate the scale of the food supply chain program.
- Optimized consensus algorithms, distributed storage mechanisms, and network optimization techniques can be employed to ensure scalability and performance.

- Ganache, a local development blockchain, provides virtual accounts with pre-defined quantities of cryptocurrency (Ethers) and can be used for testing and simulating the system's performance.

6. Training and Support:

- Adequate training and support should be provided to stakeholders to ensure the smooth adoption and utilization of the blockchain solution.
- Training programs, user manuals, and a help desk can assist users in understanding and effectively utilizing the system.
- Government officials, farmers, suppliers, distributors, and other participants should receive training on using the Remix IDE, interacting with the blockchain network, and understanding the smart contracts involved.

By implementing these security enhancements, the project aims to create a more robust and trustworthy system for food supply chain management. These measures will complement the inherent benefits of blockchain technology, such as transparency and immutability, resulting in a significantly more secure environment for all stakeholders involved in the food supply chain.

2.3.3 System Requirements

- **Ethereum:** For the simulations, an open-source blockchain technology, Ethereum, is employed. Ethereum develops decentralized apps using blockchain technology. It enables users to create smart contracts to form agreements without the intervention of a third party.
- **Remix Integrated Development Environment (IDE):** Remix is used to create, execute and test smart contracts for the blockchain-based supply-chain network. Solidity is the language used in remix to create smart contracts.

- **Ganache:** Ganache gives virtual accounts with a pre-defined quantity of cryptocurrency in Ethers which gets deducted after each transaction. Each Ganache account has its own private key and address.
- **Metamask:** It is a browser plugin that works as a gateway between Ganache and Remix IDE, allowing them to communicate.
- **The system specifications are as follows:** Intel Core i5 8th Generation, 2.4 GHz processor, 8 GB RAM, and 1.5TB storage.

Chapter 3

RELATED WORK

3.1 Existing Systems

3.1.1 AgriBlockIoT

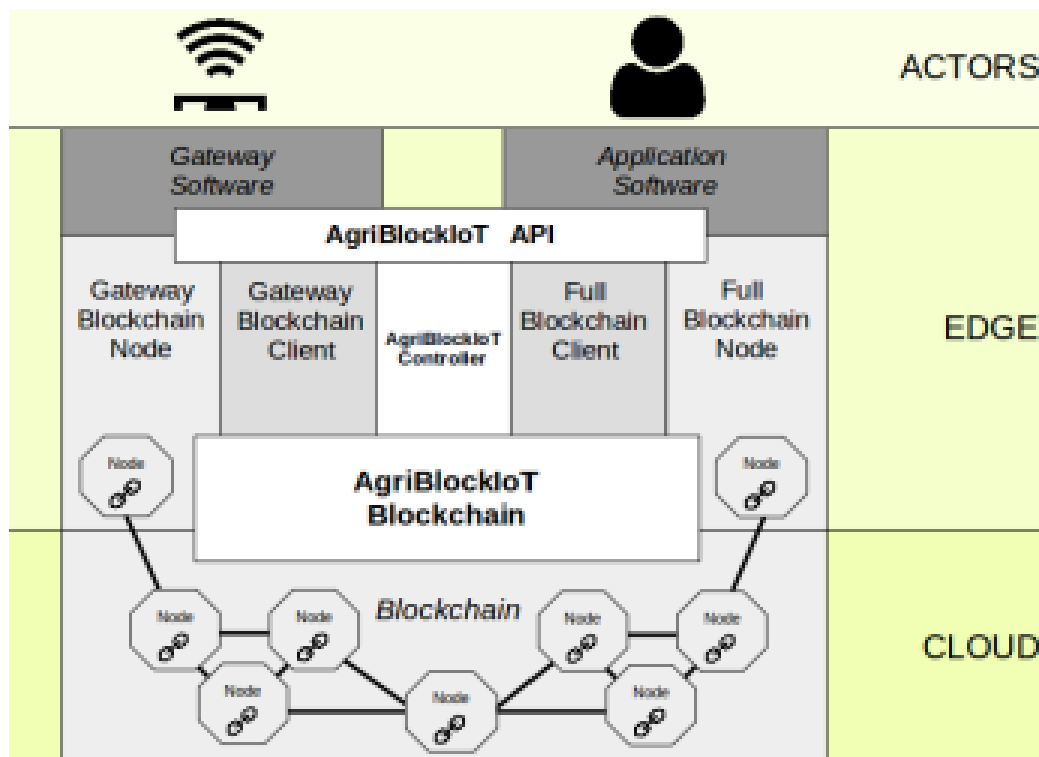


Figure 3.1: Layered architecture of AgriBlockIoT [17]

AgriBlockIoT is a fully decentralized, blockchain-based traceability solution for the Agri-Food supply chain management. It integrates IoT devices that produce and consume critical data along the supply chain, and stores this data directly on the underlying blockchain [17],

[18]. All participants, including IoT devices, must be registered users of the blockchain and have the required public/private key pairs to digitally sign each action on the distributed ledger [17], [18]. This ensures transparent, immutable, and auditable asset tracking in a trustless environment [17], [18]. However, no discussion on the performance study of the proposed framework using more limited hardware designs to test its suitability for real-world IoT devices and gateways along the Agri-Food supply chain [17], [18].

3.1.2 Monitoring Crop Prices

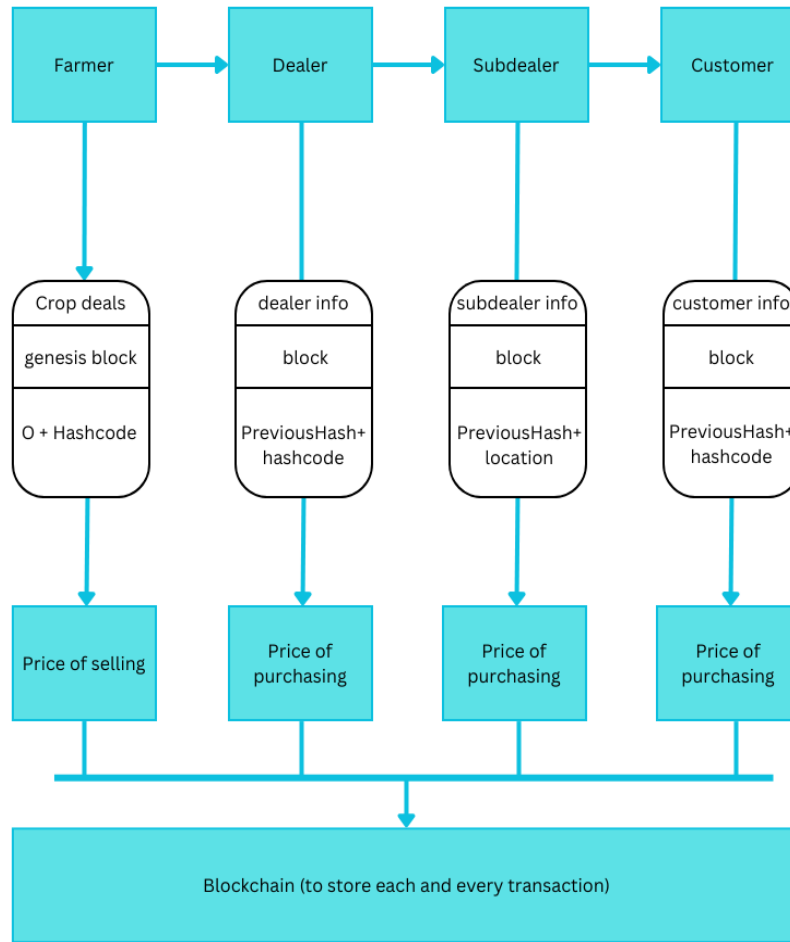


Figure 3.2: Proposed Framework of Monitoring Crop Price [19]

The framework uses blockchain technology to address traceability issues and control malpractices in the existing rice supply chain system [19], [20], [21]. It is built on an Ethereum smart contract and proposes a "Rice Coin" (RC) to control rice prices [19], [21]. This allows farmers and stakeholders to obtain fair wages by regulating prices [19], [21]. The framework includes an economic model, token distribution, conversion fees, and a layered architecture with interface, application, business logic, trust, and blockchain layers [19], [21]. This enables automated payments and delivery proof using cryptocurrency via smart contracts for successful product distribution [19], [20], [21]. However, the approach does not address challenges such as identity authentication, enforcement, scalability, privacy standards, and legislation [19], [20], [21]. Additionally, it does not utilize QR codes for item identification, and there is no real-time application implementation [19], [20], [21].

3.1.3 Reputation System

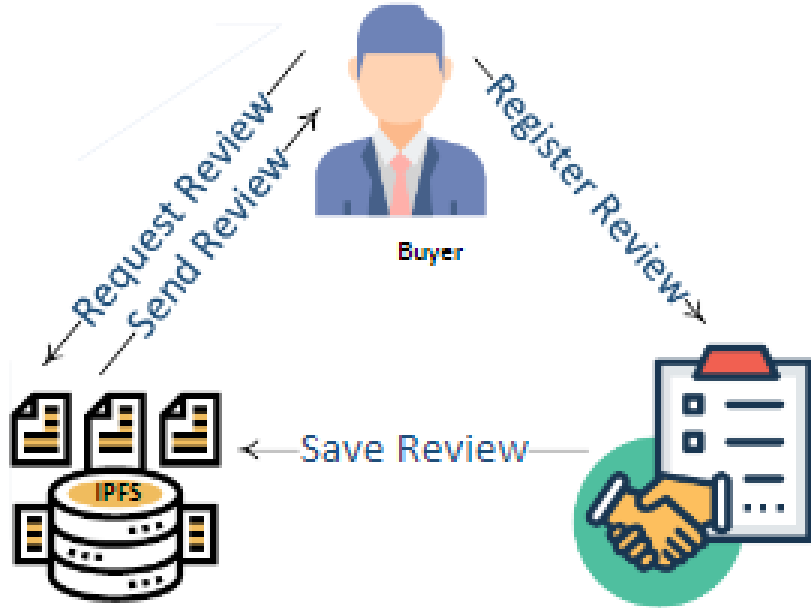


Figure 3.3: Reputation System [22]

A blockchain-based reputation system has been implemented in the Agri-Food supply chain using Ethereum smart contracts. This system aims to provide trust across trade entities by ensuring buyers are aware of the seller's reputation before engaging in a transaction [22]. When entities sign a smart contract for a transaction, a reputation smart contract is initiated. This reputation smart contract provides reviews of available data sellers to the buyer [22]. The system maintains the credibility of trade entities and product quality ratings. Since it is built on blockchain, the recorded reviews are immutable and their integrity is ensured [22]. The paper provides simulation results showing the system requires a set amount of waiting time to mine the transaction, but this waiting time is independent of the input length of reviews [22]. However, the paper does not discuss how to improve the efficiency of the proposed solution using performance parameters [22].

3.1.4 Proof of Delivery

The proposed blockchain-based proof of delivery (PoD) solution uses Ethereum smart contracts to verify the delivery of physical items between a seller and buyer, regardless of the number of transporters involved. This system ensures that all participating entities, including the seller, buyer, and transporters, act honestly and have a secure, dual-trust mechanism. The solution includes an automated ether payment system to ensure each party receives the appropriate amount when the delivery is successfully completed. Additionally, the system has a built-in arbitration mechanism to handle any disputes that may arise during the shipping process. This cost-effective approach is suitable for both domestic and international trading of physical goods. However, the PoD functionality is not available for the sale of digital assets. Overall, the system provides widespread, trustworthy, and reliable transportation, as well as automated billing, for all types of physical and virtual property exchanges [23], [24].

3.1.5 Blockchain Technology in Government Services

The research examines the use of blockchain technology in government services, including its applications, security features, and limitations. It highlights the potential benefits

of blockchain, such as its consensus algorithm, which can enable more decentralized, cost-effective, and voluntary delivery of government services. However, the research also acknowledges some drawbacks, such as a lack of emphasis on security enhancements and challenges in applying blockchain in Internet of Things (IoT) environments. Overall, the research suggests that while blockchain has significant potential for smart government services, there are still some issues that need to be addressed [25], [26].

3.2 Background

Previously, I worked on a web DAPP called Foodereum [27]. The proposed model was a three-layered architecture with supply chain entities in the first layer, a smart contract and notification system in the second layer, and a simulated Ethereum blockchain in the third layer.

The shipping details which are generated from the first layer through the supply chain entities are stored in a ledger on the second layer, through a smart contract, the shipping events are then recorded on the blockchain in the third layer. The transactions which are stored in the ledger at the second layer contain the hashes of the blocks, which helps in efficient storage management.

At second layer, a smart contract is employed which holds the core functionalities, and a strict control access mechanism is also integrated based on the roles of the supply chain entities, which determines who has access to read or write the data on the blockchain.

A notification system is also a part of second layer, which is accounted for establishing communication and enabling trust between entities. This feature also acts as a reputation system, which is in charge of handling discrepancies or miscommunications between entities.

The third layer in the system is responsible for storing the data in the blockchain which is produced from layer two. **Layer 1** consists of the several supply chain entities who communicate via the smart contract and act according to their roles. They are:

- **Sanctioner:** The only person who has the authority to sanction food commodities to

the Consumers. Thus, a genesis block will be mined at the time of registering the food commodities in the system.

- **Region-wise Distributors:** The distributors are in charge of authenticating the shipments at each level which tends to record accurate data upon receiving the physical package and transporting the package from place to place.
- **Consumers:** The end users who receives the food commodities.

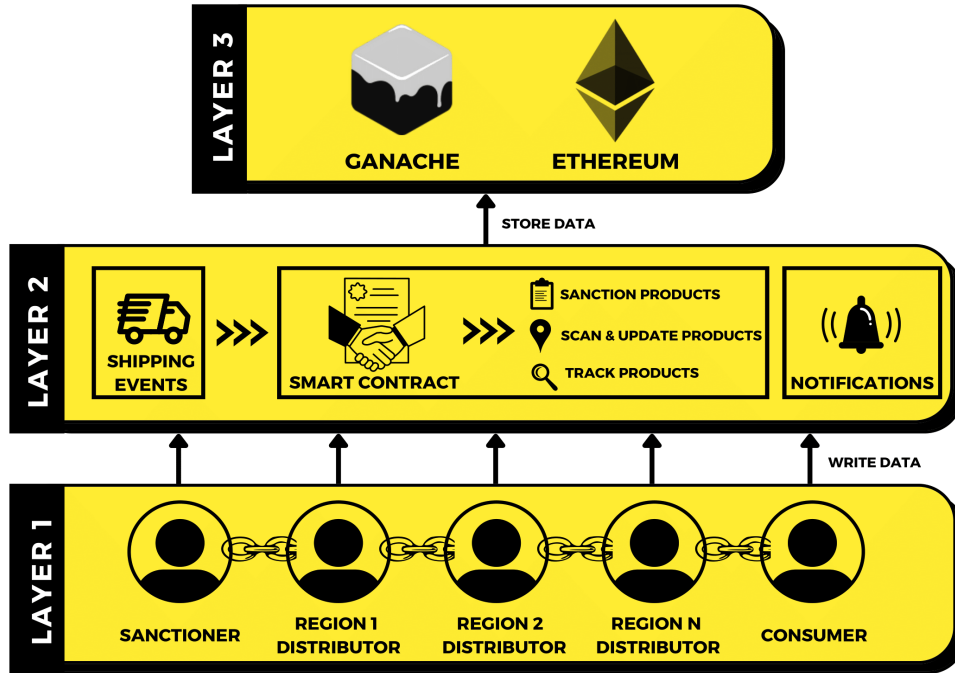


Figure 3.4: System Model [27]

Layer 2 consists of shipping events, a smart contract, and a notification system. As the sanctioner registers the food commodities to the consumers, the commodities have to be shipped from place to place. In that context, the shipping events in layer 2 are the details of every shipment which are authenticated by the distributors at their region across the supply

chain. These shipping events are then recorded on the blockchain which can be accessed by the entities to track the shipment status of the food commodities.

A self-executing Smart contract which is present in layer 2 of our system, contains three prominent functions that enable the entities to interact in the supply chain. The three functions are ¹:

Sanction Products (): In Sanction Product, the sanctioner can sanction food commodities as a single block (Genesis Block). The function accepts the name of the product and its quantity to be sanctioned as the input. Following sanctioning, a QR code and its associated Product ID will be generated, which may be downloaded and used to trace the product throughout the network. Once the food goods have been approved, users must be informed of the details through an inbuilt notification system. This makes tracking the food products transparent and efficient.

Scan & Update Products (): The Scan & Update Product feature allows the user to authenticate food products by updating the shipment status and the product's Geo-location by either scanning the QR code or entering the product ID. This function takes Product ID and the Geo-location as the inputs. Using QR code, the user can automatically update the shipment details without having to enter it manually. The Scan Update feature provides a cost-efficient, paper-free, and a time-saving alternative to traditional inventory management systems.

Track Products (): The Track Product feature allows the user to track the status of food products that have been updated using the Scan & Update Product feature. This function accepts the Product ID as the input. Each food product may be traced in detail at each level by scanning the QR code or entering the Product ID, details include entities engaged in previous shipments, date, time, and geolocation. In the event of a quality issue, the product may be tracked from its point of origin and located in no time. This feature ensures complete traceability and transparency in the system.

¹Screenshots of the following functions are attached in Appendix B

A **Notification system** is also a part of the second layer, which is accounted for establishing communication and enabling trust between entities. The Notification system also acts as a reputation system which is in charge of handling discrepancies or miscommunications between entities as any entity can communicate with any other entities in the chain to solve or raise issues. This feature also helps Food Sanctioners to ensure the authenticity and reliability of their supplies by tracking tampering attempts as the food commodities flow through the supply chain.

At **Layer 3**, the data or the shipping events that are produced by the supply chain entities from layer two are stored in the blockchain against their roles/profiles. The transactional data which are stored in the ledger at the second layer contain the hashes that point to the blocks of the Ethereum blockchain, these transactional data are stored as metadata to overcome the limitations of a storage unit, this helps in efficient storage management. Due to the incorporation of a strict control access mechanism in the system, the access to read or write the data on the blockchain based on the roles of the entities ensures that the network's privacy and confidentiality are protected.

However, there was no formalized security testing and it lacked the following features: The project initially lacked several critical security features, which left it vulnerable to various attacks. These shortcomings included:

1. **Inadequate Password Hashing:** Passwords were stored in the database without proper hashing, making user credentials susceptible to exposure in the event of a data breach.
2. **Hardcoded Credentials:** Sensitive information, such as database credentials, was hardcoded into files like `connectdb.php` and `config/Database.php`, posing a significant security risk if these files were accessed or exposed.
3. **Lack of Comprehensive Penetration Testing:** The project had not undergone thorough penetration testing, leaving it vulnerable to common web application attacks

such as SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF).

4. **Absence of Vulnerability Scanning:** There was no systematic scanning for vulnerabilities in the dependencies, libraries, and frameworks used, which could lead to the use of outdated or insecure components.

These were some deficiencies that collectively weakened the security posture of the project, making it susceptible to various forms of cyberattacks and data breaches.

In this capstone project, we identified relevant best practices documents and tools, which we then applied to the Foodereum project to examine and resolve its security issues.

Chapter 4

METHODOLOGY

In this capstone project, well-established procedures and methodologies for testing software, web applications, and decentralized applications (dApps) for security properties were identified and utilized. The primary focus was on examining and addressing security issues in the Foodereum project. The methodologies included static analysis, dynamic analysis, and vulnerability assessments based on best practices recommended by leading security organizations such as OWASP.

4.1 Selection of Methodologies and Tools:

In developing our methodology, we carefully considered various approaches and tools available for security testing and analysis. Our selection was based on several factors:

1. **Relevance to blockchain and smart contract security:** Given the unique nature of blockchain-based applications, we prioritized tools and techniques specifically designed for smart contract analysis and blockchain security.
2. **Industry recognition and effectiveness:** We chose methodologies and tools widely recognized in the cybersecurity community for their effectiveness in identifying vulnerabilities.
3. **Compatibility with Ethereum-based systems:** Since Foodereum is built on Ethereum, we selected tools compatible with Solidity and the Ethereum Virtual Machine (EVM).

4. **Coverage of different security aspects:** We aimed to cover a wide range of potential vulnerabilities, from code-level issues to runtime behaviors.

4.2 OWASP Guidelines and Best Practices:

The Open Web Application Security Project (OWASP) guidelines and best practices were heavily relied upon for enhancing the security of the Foodereum blockchain-based food supply chain authentication system. Specifically, we focused on:

1. The OWASP Top 10 and the OWASP Application Security Verification Standard (ASVS) as structured frameworks for identifying and addressing common web application vulnerabilities.
2. Key OWASP best practices applied include:
 - **Input Validation and Output Encoding:** To prevent injection attacks.
 - **Proper Authentication and Session Management:** Implementing strong password policies and two-factor authentication.
 - **Securing Data Storage and Transmission:** Ensuring encryption of sensitive data.
 - **Principle of Least Privilege:** Restricting access rights for users to the bare minimum necessary.
 - **Regular Security Updates and Patch Management:** Keeping the system up-to-date with the latest security patches.
3. Used OWASP ZAP (Zed Attack Proxy) for dynamic analysis, citing its robust feature set and active open-source community.
4. The methodology focused on addressing common vulnerabilities outlined in the OWASP Top 10 for smart contracts and decentralized applications, including:

- **Access Control:** Ensuring proper implementation of role-based access control in smart contracts.
- **Reentrancy:** Protecting against reentrancy attacks, a common vulnerability in smart contracts.
- **Arithmetic Issues:** Checking for integer overflow/underflow vulnerabilities.
- **Dependency Management:** Assessing and securing external contract dependencies.
- **Front-Running:** Implementing measures to prevent transaction ordering exploitation.

By incorporating these OWASP guidelines and best practices, the team aimed to create a more robust and secure system for food supply chain management, addressing potential vulnerabilities at multiple levels throughout the application.

4.3 Additional Techniques Considered:

Based on the information provided in the project report, there were several additional techniques that could have been considered for enhancing the security of the Foodereum blockchain-based food supply chain authentication system:

1. **Formal Verification:** This technique could have been used to provide mathematical proof of contract correctness. However, it was likely not implemented due to its time-consuming nature.
2. **Fuzzing:** While effective for finding edge cases and unexpected inputs, fuzzing was considered but not implemented in this phase, possibly due to time constraints.
3. **Symbolic Execution:** This powerful technique for smart contract analysis was considered but may have been deemed too resource-intensive for the project's scope.

4. **Threat Modeling:** Although valuable for a holistic security approach, threat modeling was considered but not implemented, as the focus was on immediate, actionable improvements to the existing system.
5. **Runtime Application Self-Protection (RASP):** This technique for real-time attack mitigation was considered but required further evaluation for the specific use case of Foodereum.
6. **Property-based Testing:** The report mentions exploring Echidna for property-based testing, but it was ultimately not used as the chosen dynamic analysis approach was deemed to provide better coverage for their specific use case.

These additional techniques were considered but not implemented due to various factors such as project timeline, resource constraints, or the specific requirements of the Foodereum system. The chosen methodology focused on static analysis, dynamic analysis, and penetration testing as the primary security assessment techniques.

4.4 Tool Selection Rationale:

We carefully selected a range of security testing tools based on their effectiveness, industry reputation, and suitability for blockchain-based applications:

1. **SonarQube:** Chosen for its ability to perform in-depth static code analysis and identify code quality issues. Its support for multiple programming languages, including Solidity, made it ideal for analyzing smart contracts.
2. **OWASP ZAP (Zed Attack Proxy):** Selected for dynamic analysis due to its comprehensive set of security testing features and its open-source nature, allowing for customization to suit our specific needs.
3. **Mythril:** A specialized tool for Ethereum smart contract analysis, chosen for its ability to detect security vulnerabilities specific to blockchain applications.

4. **Truffle Suite:** Used for its integrated development environment and testing framework specifically designed for Ethereum smart contracts.
5. **Snyk:** Incorporated for its powerful penetration testing capabilities and its ability to scan for vulnerabilities in both application code and dependencies. Snyk's integration with our development workflow allowed for continuous security monitoring throughout the development process.

4.5 Static Analysis Enhancements:

1. **Password Hashing:** To enhance password security, we implemented hashing on the login.php page. Using algorithms like bcrypt, passwords are now hashed before being stored in the database. This measure helps protect user credentials in case of a data breach.
2. **Removal of Hardcoded Credentials:** Identified hardcoded credentials in the connectdb.php and config/Database.php files. These credentials were replaced with environment variables and secure methods of handling sensitive information, thereby reducing the risk of credential exposure.

4.6 Dynamic Analysis Enhancements:

1. **Penetration Testing:** Comprehensive penetration testing was conducted to identify vulnerabilities such as SQL injection, XSS, and CSRF. Tools like OWASP ZAP and Burp Suite were used to simulate attacks. These tests revealed several flaws:
 - SQL injection vulnerabilities were found in several input fields, allowing attackers to manipulate database queries.
 - XSS vulnerabilities were identified on multiple pages where user input was improperly sanitized.

- **CSRF vulnerabilities** were detected, potentially allowing unauthorized actions on behalf of authenticated users.

2. **Vulnerability Scanning:** Automated vulnerability scanning was performed on the dependencies, libraries, and frameworks used in the project. Tools like Dependency-Check and Retire.js were employed to ensure all components were up-to-date and free from known vulnerabilities. The scans highlighted outdated libraries with known security issues, prompting immediate updates and patches.

4.7 *Demonstration of Testing and Findings:*

The testing processes were systematically executed as follows:

1. **Static Analysis:**

- **Implementation:** The source code was reviewed, and improvements were made by integrating security best practices such as password hashing and the removal of hardcoded credentials.
- **Findings:** Initial analysis showed plain text passwords and hardcoded credentials, which were subsequently mitigated.

2. **Dynamic Analysis:**

- **Penetration Testing:** Simulated attacks uncovered several critical vulnerabilities:
 - **SQL Injection:** Found in user input fields, leading to potential unauthorized database access.
 - **XSS:** Detected on pages where user inputs were rendered without proper sanitization.
- **Vulnerability Scanning:** Regular scans revealed outdated libraries that were promptly updated to their secure versions.

4.8 *Penetration Testing*

The purpose of penetration testing on a web application is to identify vulnerabilities and weaknesses in the application's security, assessing the risk of a real-world attack and potential impact on sensitive data.

SYNK penetration testing is used for this purpose on the web application to scan and test exploitation that involves simulating a cyber-attack on a web application to identify vulnerabilities and weaknesses. This includes:

- Identifying sensitive data entry points (e.g., login forms, search bars)
- Testing for SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF)
- Analyzing input validation and sanitization
- Scanning for vulnerabilities in dependencies, libraries, and frameworks
- Attempting to bypass security controls (e.g., CAPTCHA)
- Exploiting identified vulnerabilities to gain unauthorized access or control

By conducting a web application penetration test, we can pinpoint areas where their application is vulnerable, and take action to remediate issues before malicious actors can exploit them, ultimately protecting their users' data and maintaining trust in their online presence.

By integrating Snyk into our penetration testing process, we were able to achieve more comprehensive security coverage. Snyk's ability to scan both our application code and its dependencies allowed us to identify and address vulnerabilities that might have otherwise gone unnoticed. The tool's integration with our development workflow enabled us to catch

and fix security issues early in the development cycle, significantly improving the overall security posture of the Foodereum project.

This comprehensive approach, combining various tools and methodologies, allowed us to identify and address potential vulnerabilities across the entire system, from smart contracts to the user interface, ensuring a more secure and reliable food supply chain authentication system.

4.9 Challenges and Unsuccessful Approaches:

During the security testing process, we encountered several challenges and explored approaches that ultimately proved less effective:

1. **Manual Code Review:** Initially, we attempted a comprehensive manual code review of the smart contracts. However, this proved to be time-consuming and prone to human error, leading us to prioritize automated tools for initial vulnerability detection.
2. **Fuzz Testing:** We experimented with fuzz testing tools to identify potential vulnerabilities in the smart contracts. While promising, many of these tools were still in early development stages and produced numerous false positives, making them less reliable for our purposes.
3. **Blockchain-specific Security Scanners:** We evaluated several blockchain-specific security scanners, but found that many lacked the depth of analysis required for our project. This led us to combine general-purpose security tools with blockchain-specific ones for a more comprehensive approach.

Chapter 5

IMPLEMENTATION

5.1 *Implementation Platforms*

5.1.1 *Hardware*

The hardware requirements for this project consist of a powerful Intel Core i5 8th Generation processor clocked at 2.4 GHz, providing ample processing power for the tasks at hand. The system is equipped with 8 GB of RAM, ensuring smooth multitasking and efficient handling of data. For storage, a spacious 1.5 TB drive is available, allowing for the storage of large amounts of data and resources required for the implementation.

5.1.2 *Software*

- **Operating System:** Windows (10) (64bit) or MacOS
- **Software Used:** Ganache, Metamask, Remix IDE, MAMP
- **Programming Languages:** Solidity
- **Server:** Apache

5.2 *Implementation Details*

5.2.1 *Backend Integration*

Ganache

Ganache is an essential tool for Ethereum development, offering a personal blockchain for deploying, testing, and debugging smart contracts and decentralized applications (dApps).

As part of the Truffle Suite, Ganache provides developers with a local Ethereum environment, enabling fast and efficient iteration without the need for real Ether or gas. It features both a graphical user interface (GUI) and a command-line interface (CLI), offering flexibility for different development needs. Key functionalities include instant mining, customizable gas fees, and adjustable block times, which allow developers to control their testing conditions precisely. By facilitating detailed inspection of blockchain state, transactions, and events, Ganache ensures thorough testing and robust smart contract development.

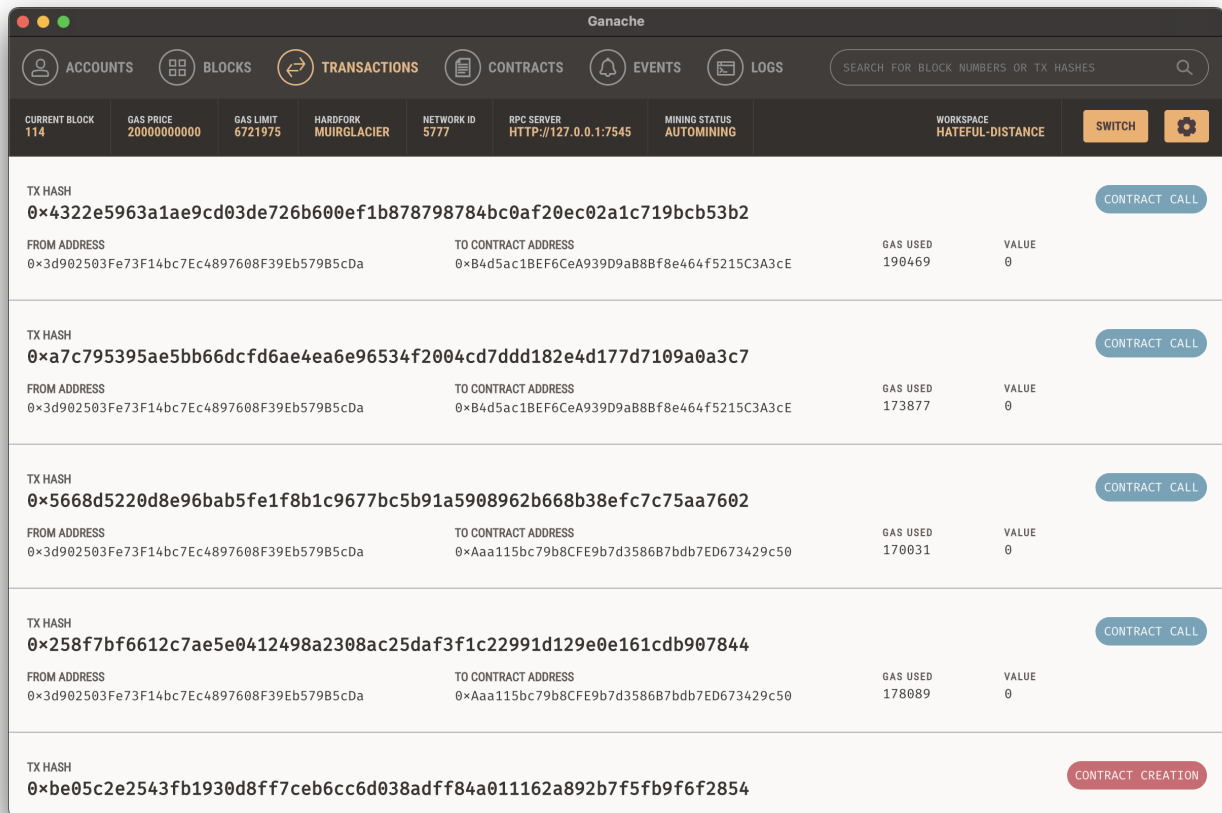


Figure 5.1: Ganache Transactions

Metamask

MetaMask is a widely used cryptocurrency wallet and gateway to decentralized applications (dApps) on the Ethereum blockchain. Available as a browser extension and mobile app, MetaMask allows users to manage their Ethereum accounts, securely store private keys, and interact with dApps directly from their web browser or smartphone. It simplifies the process of sending and receiving Ether and ERC-20 tokens, providing an intuitive interface for managing transactions and account balances. MetaMask also features an integrated token exchange service and supports multiple networks, including custom ones. By serving as a bridge between users and the decentralized web, MetaMask enhances accessibility and usability for both novice and experienced users, making it an essential tool for engaging with the Ethereum ecosystem.

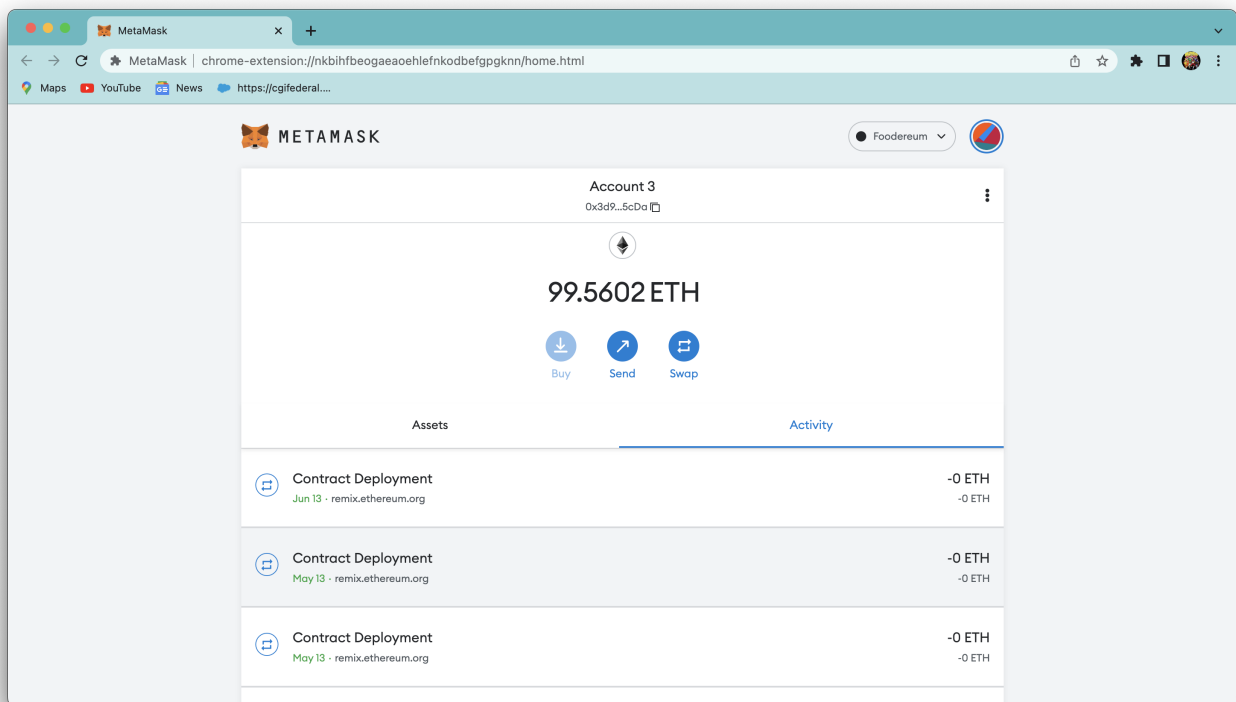


Figure 5.2: Metamask with Personalized Network

Remix IDE

Remix IDE is a powerful, web-based integrated development environment (IDE) specifically designed for writing, testing, and deploying smart contracts on the Ethereum blockchain. It offers a comprehensive suite of tools and features that streamline the development process, including a built-in compiler, debugger, and static analysis tools. Remix supports Solidity, the primary programming language for Ethereum smart contracts, providing real-time code analysis and highlighting potential issues as developers write code. The IDE's intuitive interface allows for easy deployment and interaction with smart contracts on various networks, including local, test, and main Ethereum networks. Additionally, Remix integrates with other Ethereum tools like MetaMask and Truffle, enhancing its functionality and making it an indispensable tool for both novice and experienced blockchain developers.

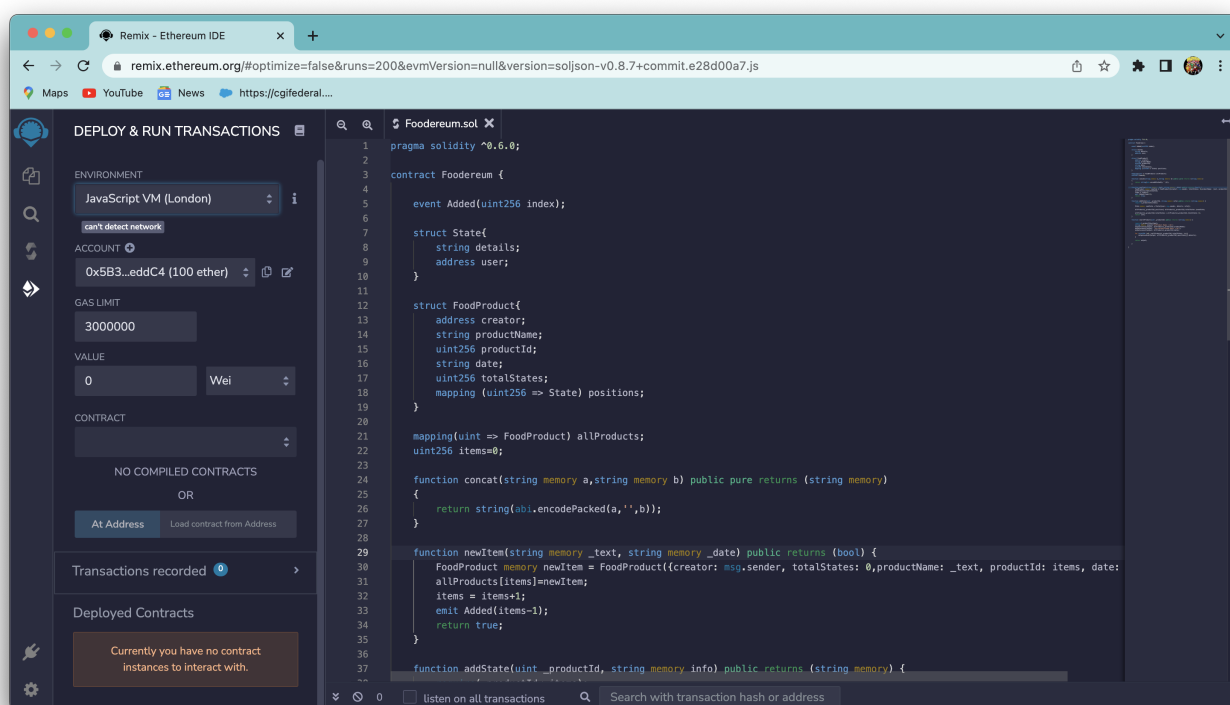


Figure 5.3: Remix IDE

SYNK Penetration Testing

SYNK penetration testing is a type of penetration testing that focuses on identifying vulnerabilities and weaknesses in computer systems, networks, and web applications. The term "SYNK" stands for "SYstematic Network Knowledge", which refers to the methodical and comprehensive approach used in this type of testing.

The goal of SYNK penetration testing is to provide a complete picture of the system's or network's security posture, highlighting areas that need improvement and providing recommendations for remediation. This type of testing is essential for organizations to protect themselves against cyber threats and maintain the confidentiality, integrity, and availability of their systems and data.



Figure 5.4: SYNK Penetration Testing

Chapter 6

RESULTS

In this chapter, the results of the improved security aspects after analysis and testing are done on the Foodereum web project. Also, a detailed analysis of Gas Consumption by each Smart Contract & its Functions is discussed.

The security enhancements applied to the Foodereum project resulted in significant improvements across various aspects of the system. The results of the static and dynamic analyses, along with the subsequent remediation efforts, are detailed below.

6.1 Static Analysis Results:

1. **Improved Password Hashing:** By integrating bcrypt for password hashing on the login.php page, the security of user credentials was significantly strengthened. Previously stored plain text passwords were rehashed and securely stored, mitigating the risk of exposure in the event of a data breach.
2. **Elimination of Hardcoded Credentials:** The removal of hardcoded credentials from the connectdb.php and config/Database.php files marked a crucial improvement. These credentials were replaced with environment variables and secure configuration methods, ensuring sensitive information is handled securely. This change reduces the risk of accidental exposure and enhances overall system security.

6.2 Dynamic Analysis Results:

1. **Penetration Testing:** Penetration testing revealed several critical vulnerabilities, which were systematically addressed:

- **SQL Injection:** Vulnerabilities were discovered in several input fields. By implementing parameterized queries and prepared statements, these vulnerabilities were mitigated, preventing unauthorized database access and manipulation.
 - **Cross-Site Scripting (XSS):** Multiple pages were found to be vulnerable to XSS attacks. The implementation of robust input validation and output encoding significantly reduced the risk of malicious script injection, thereby protecting users from data theft and session hijacking.
2. **Vulnerability Scanning:** Automated scans of the project's dependencies, libraries, and frameworks identified several outdated components with known security issues. Prompt updates and patches were applied to ensure all components were up-to-date and secure. This proactive measure minimized the attack surface and aligned the project with current security standards.

6.3 Summary of Enhancements:

The security enhancements were systematically implemented, focusing on critical areas such as login mechanisms, database connections, and the overall handling of user inputs and data storage. The following improvements were achieved:

1. **Enhanced User Credential Security:** By hashing passwords before storage and eliminating hardcoded credentials, user data is better protected against breaches and unauthorized access.
2. **Improved Input Handling and Validation:** Implementing robust input validation and output encoding mitigated XSS vulnerabilities, enhancing the security of user interactions with the system.

6.4 Areas of Inconclusive Testing:

While most tests provided clear insights, some areas were inconclusive:

1. Advanced Persistent Threats (APTs):

- **Limitations:** The project did not encompass exhaustive testing for APTs due to their complex and sophisticated nature, often requiring continuous monitoring and advanced threat detection systems.

2. Zero-Day Vulnerabilities:

- **Challenges:** Identifying zero-day vulnerabilities remains inherently challenging as these are unknown to the security community at the time of testing. Continuous updates and vigilance are required to mitigate such risks.

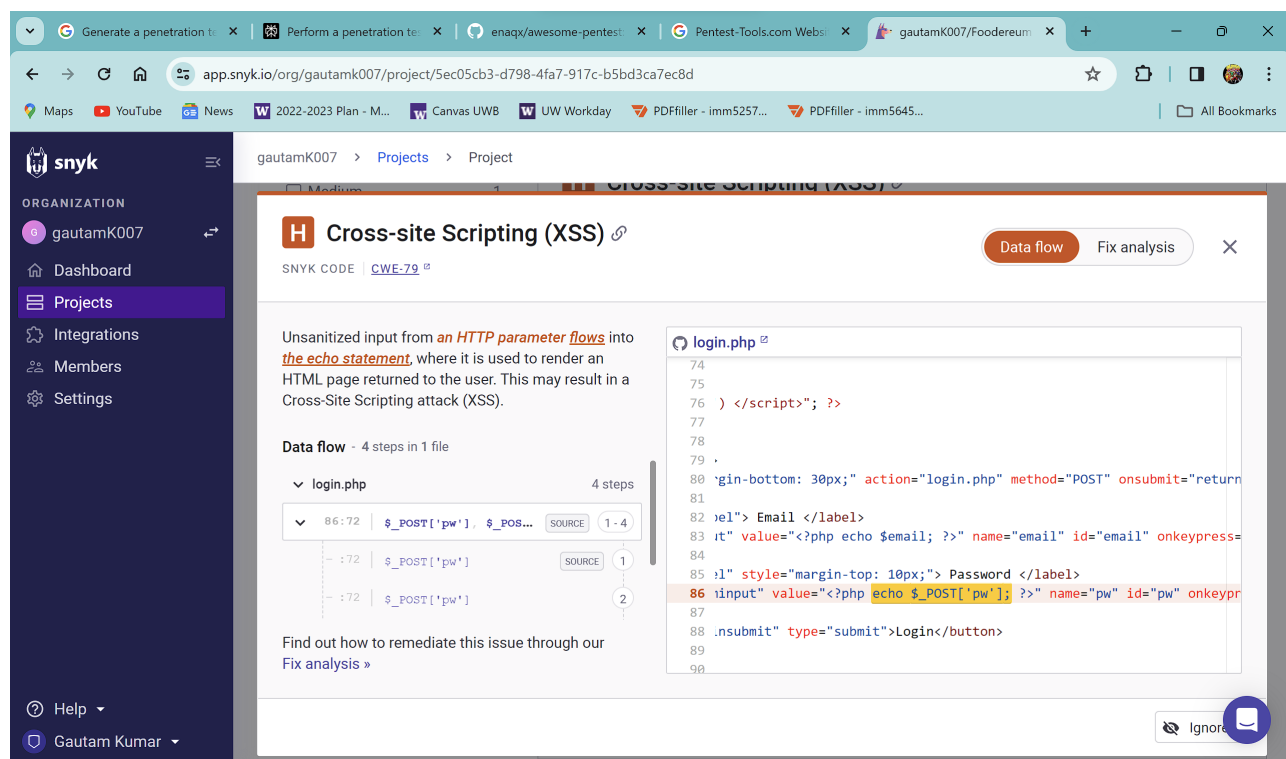


Figure 6.1: XSS on login.php page

Figure 6.1 shows an XSS attack on the login.php page.

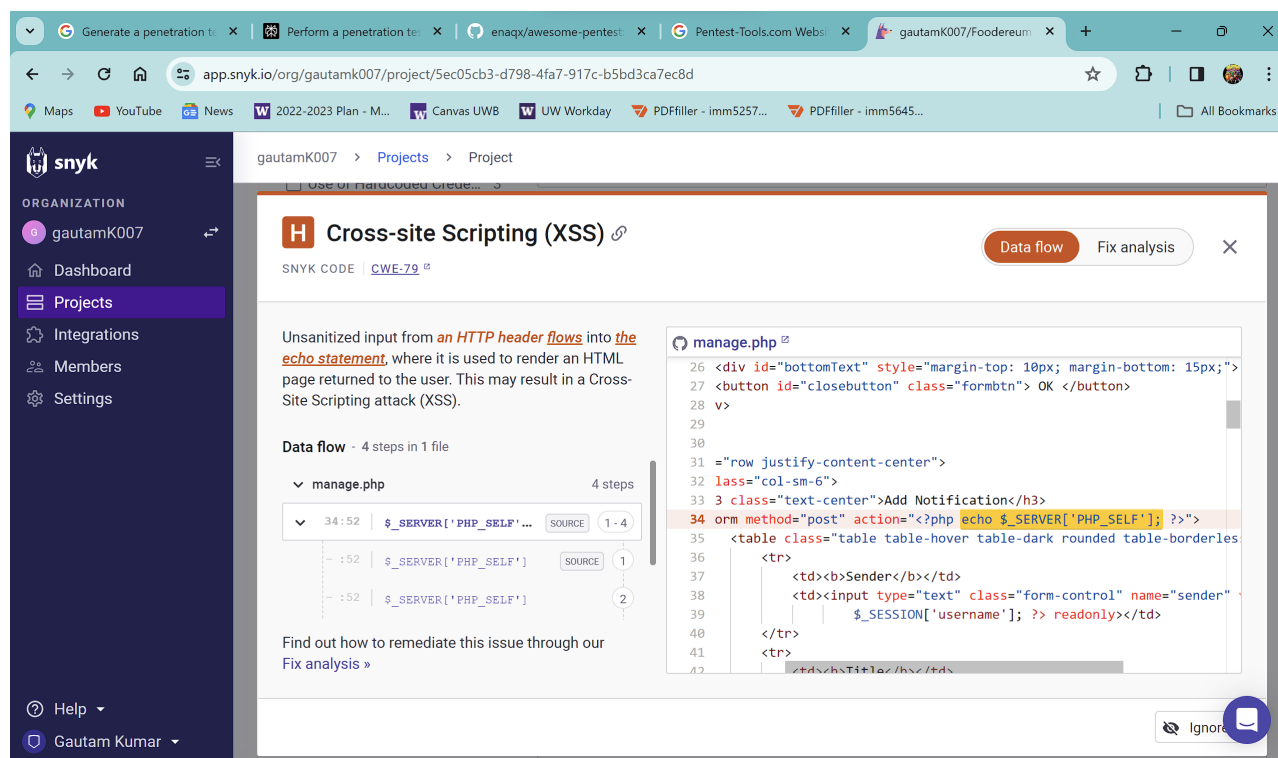


Figure 6.2: XSS on manage.php page

Figure 6.2 shows an XSS attack on the manage.php page.

Figures 6.1 and 6.2 illustrate XSS (Cross-Site Scripting) attacks on two different pages of a web application. Figure 6.1 shows an XSS attack targeting the login.php page, where malicious scripts could be injected into the login form, potentially compromising user credentials and session information by executing arbitrary scripts in the context of the user's browser. Similarly, Figure 6.2 demonstrates an XSS attack on the manage.php page, which could allow attackers to manipulate management functions or steal sensitive data through the execution of unauthorized scripts. Combining these two scenarios highlights the broader vulnerability of the web application to XSS attacks, underscoring the need for comprehensive security measures such as input validation, output encoding, and proper use of security functions like htmlspecialchars() in PHP to mitigate such risks effectively.

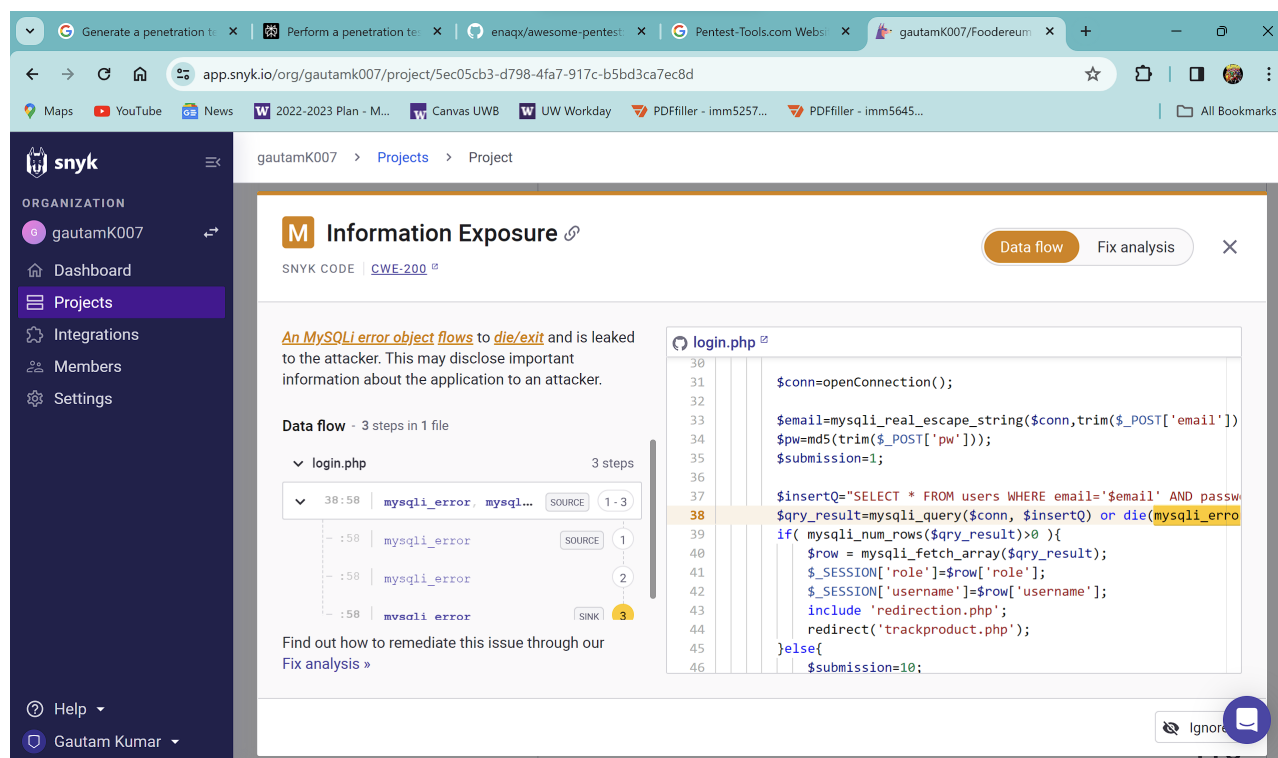


Figure 6.3: Information Exposure on login.php page

Figure 6.3 shows an Information Exposure attack on the login.php page.

Figure 6.3 illustrates an Information Exposure attack on the 'login.php' page of the Foodereum web application. This type of attack occurs when sensitive information is inadvertently exposed to users or attackers, potentially compromising the security of the system. In the depicted scenario, the 'login.php' page likely reveals critical details such as error messages, user credentials, or other confidential data due to improper handling or lack of secure coding practices. Such exposure can provide attackers with valuable information that can be exploited to gain unauthorized access to the system, leading to further breaches and data theft.

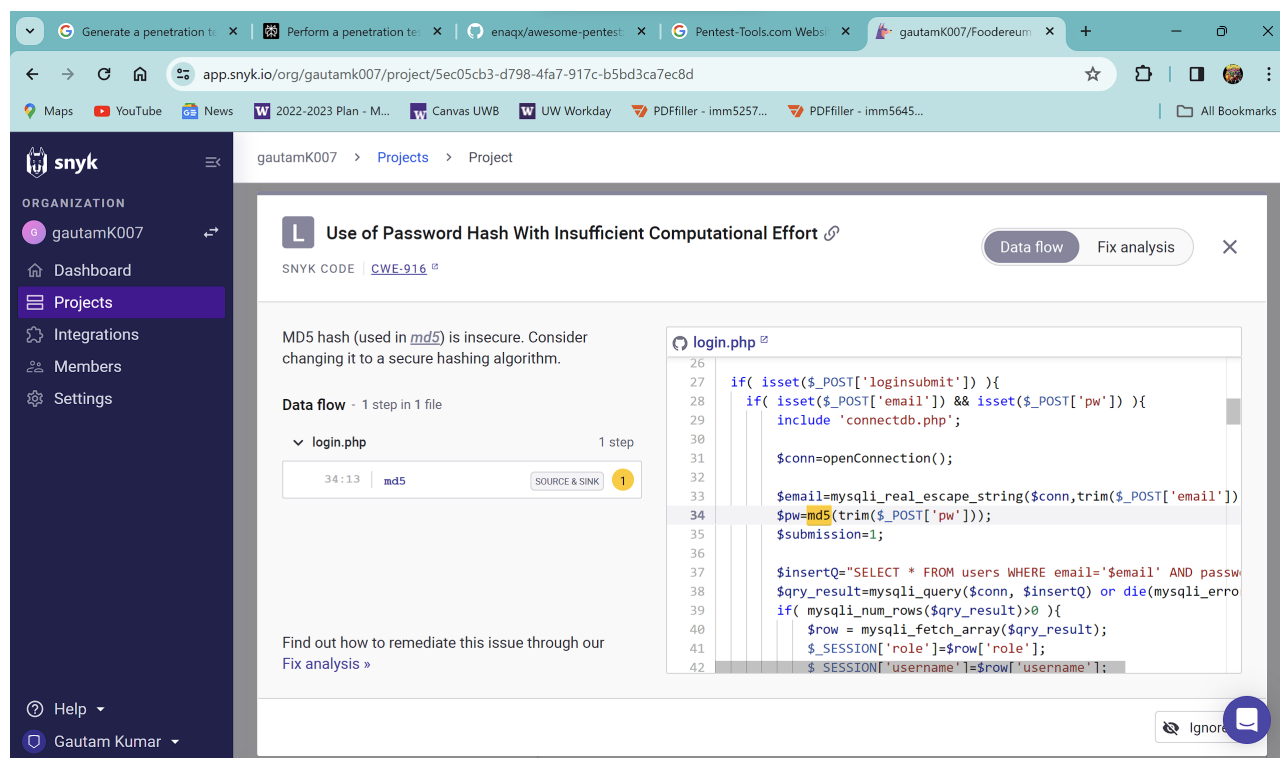


Figure 6.4: Use of Password Hash on login.php page

Figure 6.4 shows Use of Password Hash which leads to an insufficient computational effort on login.php page

Figure 6.4 highlights a vulnerability on the login.php page due to the use of password hashes, which results in insufficient computational effort during the authentication process. The inadequacy lies in the implementation of the hashing algorithm, which is not computationally intensive enough to effectively thwart brute-force attacks. This means that an attacker could potentially generate and test a large number of password guesses in a relatively short amount of time, compromising the security of user accounts. To mitigate this issue, it is recommended to use a more robust hashing algorithm such as bcrypt, scrypt, or Argon2, which are designed to be computationally expensive and resistant to such attacks.

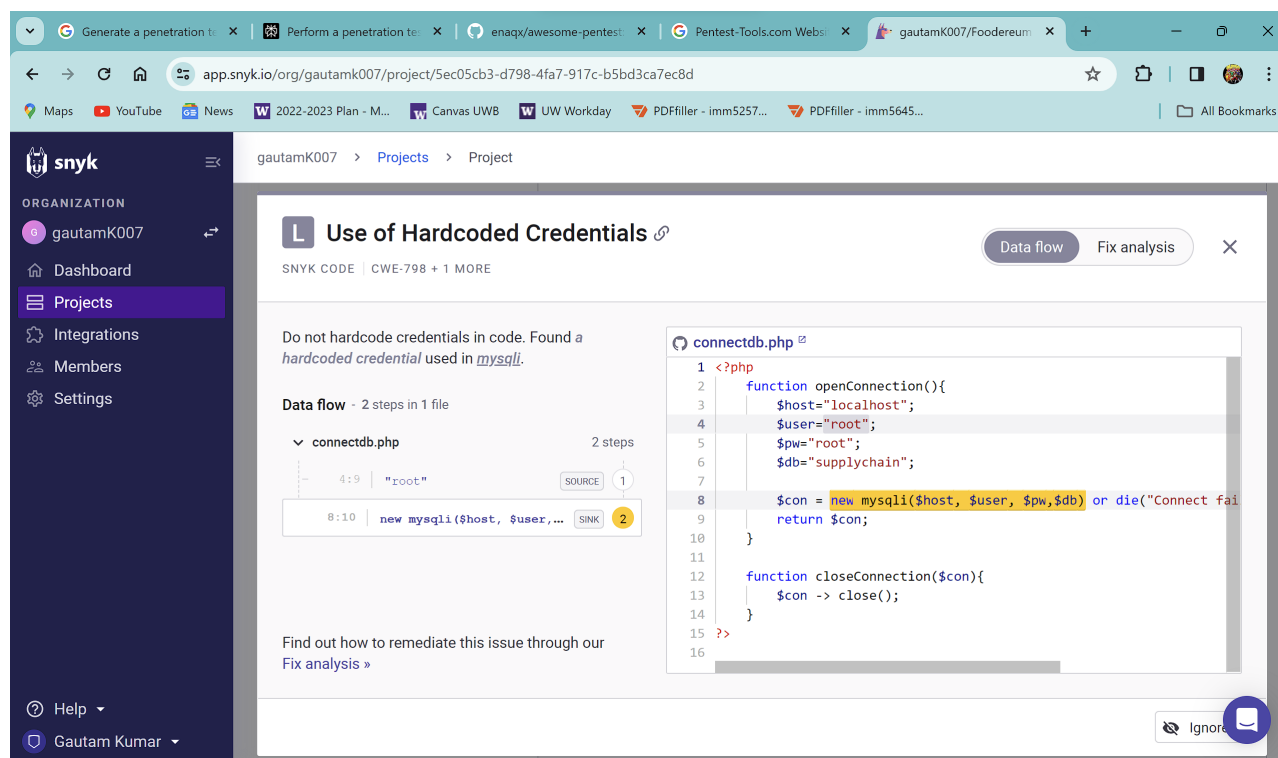


Figure 6.5: Use of Hardcoded Credentials on connectdb.php page

Figure 6.5 shows the Use of Hardcoded Credentials on the MySQL database on connectdb.php page.

Figure 6.5 illustrates the use of hardcoded credentials in the MySQL database connection on the 'connectdb.php' page. This security flaw involves embedding the database username and password directly in the source code, which can be easily exploited if unauthorized individuals access the code. Exposing these credentials can lead to unauthorized access to the database, resulting in potential data breaches and manipulation. The figure highlights this issue, underscoring the importance of using secure methods such as environment variables or secure credential management to protect sensitive information.

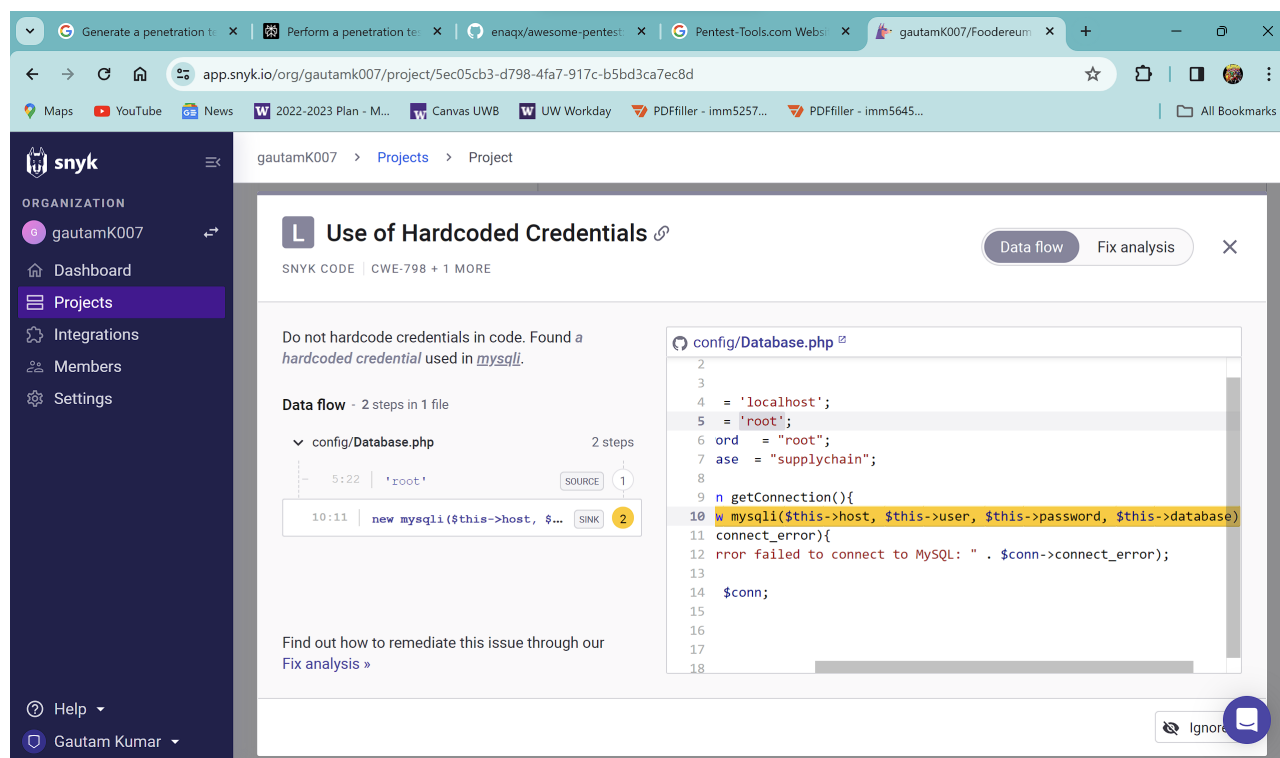


Figure 6.6: Use of Hardcoded Credentials on config/Database.php page

Figure 6.6 shows the Use of Hardcoded Credentials on config/Database.php page.

Figure 6.6 reveals the use of hardcoded credentials in the config/Database.php file, which poses a significant security risk. Hardcoding sensitive information like database credentials directly into the application code is a major vulnerability, as it exposes these credentials to anyone with access to the codebase. If an attacker gains access to the code, they can easily obtain the hardcoded credentials and potentially compromise the entire database. This practice violates the principle of least privilege and increases the attack surface of the application. To mitigate this risk, it is crucial to store database credentials securely, such as in environment variables or encrypted configuration files, and retrieve them dynamically during runtime. Additionally, implementing strong access controls and regularly rotating credentials can further enhance security.

6.5 Analysis of Gas Consumption by Smart Contract & its Functions

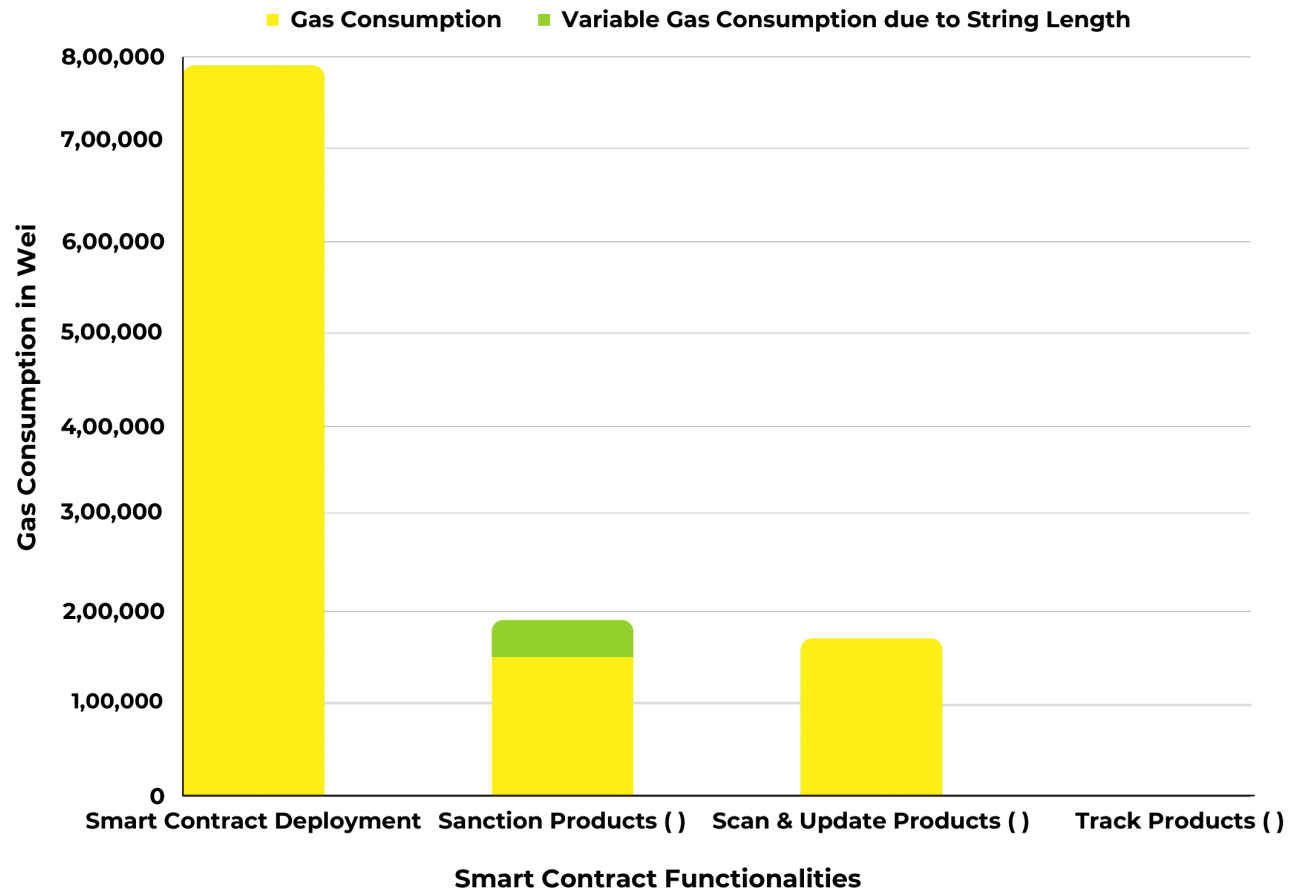


Figure 6.7: Gas Consumption vs Smart Contract and its Functions [27]

Figure 6.7 discusses the gas consumption of a smart contract and its functions on the Ethereum blockchain. Deploying a smart contract on the Ethereum blockchain consumes the maximum amount of gas (measured in Wei) compared to executing any of the contract's functions. The Foodereum project consists of three smart contract functions: Sanction Products(), Scan & Update Products(), and Track Products(). The gas consumption for deploying the smart contract is higher than the gas required to execute any of the three Foodereum smart contract functions.

The Sanction Products() in the smart contract accept the product name and its quantity as input strings. This function is responsible for registering the food commodities that are sanctioned by the sanctioner in the blockchain and then recording these products against the sanctioner's profile. It has been observed that this function consumes the highest gas value compared to the other two smart contract functions. As the input string length increases, the amount of gas spent by the Sanction Products() also increases, indicating a slight variation in gas consumption related to the string length. This suggests that the Sanction Products() is the most resource-intensive operation in the smart contract, likely due to the complex processing involved in registering the sanctioned products and updating the sanctioner's profile. The increased gas consumption with longer input strings may be a consideration for optimizing the function's efficiency, especially if the smart contract is expected to handle large volumes of data.

The Scan & Update Products() uses relatively less gas compared to the Sanction Products(). This is because Scan & Update Products() is used when shipping products to different regions, and it contains details of products that were previously sanctioned. There is not a significant amount of variability in gas consumption when this function is triggered. In other words, Scan & Update Products() for shipping products is more efficient in terms of gas usage compared to Sanction Products(), as Scan & Update Products() deals with pre-approved items rather than requiring additional checks. The gas consumption for the Scan & Update Product() is relatively stable and predictable.

The Track Products() is the only feature that is offered to all the entities in the supply chain. This feature is designed to operate without incurring any gas fees in the system. Tracking plays a crucial role in a supply chain environment, and its usage can be very frequent. Due to the importance and frequency of tracking, this feature aims to provide reliable access to the details of sanctioned products that are recorded on the blockchain. By doing so, the feature ensures complete traceability and transparency throughout the entire supply chain. The ability to reliably track and trace products is essential for maintaining trust and accountability across all stakeholders in the supply network.

Chapter 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 *Conclusion*

The current food supply chain system lacks transparency, traceability, and auditability, leading to inefficiencies such as food waste, contamination, high costs, and communication gaps between supply chain partners. To address these issues, the proposed work suggests adopting a private blockchain-based authentication system for supply chain management, specifically an Ethereum-based system for recording, tracking, and verifying food products throughout the supply chain.

The immutability of blockchain technology ensures that transactions cannot be altered or concealed, as they are recorded in a distributed ledger accessible to all authorized parties. This means that whenever a product is delivered, the new receiver can access the product's entire history on the blockchain network, providing transparency and traceability.

By implementing this blockchain-based solution, the food industry aims to improve food quality and safety, increase consumer appeal, and overcome challenges such as centralized systems, third-party involvement, high transportation and storage costs, food waste due to spoilage and contamination, lack of accountability, and communication gaps between supply chain partners.

The systematic application of static and dynamic analysis methodologies has substantially improved the security posture of the Foodereum project. By identifying and mitigating critical vulnerabilities, including those related to password hashing, hardcoded credentials, SQL injection, and XSS, the project now adheres more closely to the best practices recommended by OWASP.

The enhancements made—such as implementing robust password hashing, eliminating hardcoded credentials, conducting comprehensive penetration testing, and ensuring all dependencies are up-to-date—have fortified the project’s defenses against potential attacks. The focus on improving input validation, output encoding, and secure session management has also contributed to a more secure and reliable environment for users.

Despite some areas remaining inconclusive, such as the detection of advanced persistent threats (APTs) and zero-day vulnerabilities, the steps taken during this capstone project lay a strong foundation for ongoing security maintenance and future enhancements. The Foodereum project is now better equipped to protect user data and ensure the integrity of its blockchain-based food supply chain system. The efforts made during this project highlight the importance of continuous security assessments and the implementation of best practices to safeguard digital systems against evolving threats.

7.2 *Future Enhancements*

1. Integration of IoT Technologies:

- The integration of Internet of Things (IoT) technologies, such as sensors, cameras, and Radio Frequency Identification (RFID) tags, can significantly enhance the traceability and accountability of food products throughout the supply chain. These technologies enable real-time monitoring and tracking from production to consumption, ensuring greater transparency and reducing the risk of contamination and fraud.

2. Scalability of Distribution Network:

- Expanding the distribution network across multiple areas will increase the scalability and reach of the current food traceability system. This expansion allows for broader coverage and improved oversight, ensuring that more regions benefit from enhanced traceability and accountability measures.

3. Simplified Login Process:

- Simplifying the login process for the decentralized application (DApp) by integrating it with MetaMask, a popular cryptocurrency wallet, would streamline user access while providing enhanced security through its robust authentication mechanisms. This integration will make the system more user-friendly and secure.

To further enhance the security of the Foodereum project, several future enhancements have been identified. These enhancements aim to build on the current security measures and address potential vulnerabilities that may arise as the project evolves.

1. Advanced Threat Detection and Response:

- **Implementation of Intrusion Detection Systems (IDS):** Deploying IDS can help monitor network traffic for suspicious activities and potential intrusions. This system will provide real-time alerts and automated responses to mitigate threats promptly.
- **Behavioral Analysis:** Utilizing machine learning algorithms to analyze user behavior patterns can help detect anomalies and potential security breaches. This proactive approach can identify and respond to advanced persistent threats (APTs) more effectively.

2. Enhanced Authentication Mechanisms:

- **Multi-Factor Authentication (MFA):** Integrating MFA for user authentication adds an additional layer of security, making it more challenging for unauthorized users to gain access. MFA can combine passwords with biometric verification, SMS codes, or authenticator apps.
- **Decentralized Identity Management:** Implementing blockchain-based identity management systems can provide secure and tamper-proof user authentication, enhancing overall security and privacy.

3. Regular Security Audits and Penetration Testing:

- **Periodic Security Audits:** Conducting regular security audits by third-party experts will help identify new vulnerabilities and ensure compliance with the latest security standards and regulations.
- **Continuous Penetration Testing:** Implementing a continuous penetration testing program can help identify and address vulnerabilities in real-time. This approach ensures that the system remains resilient against emerging threats.

4. Improved Data Encryption:

- **End-to-End Encryption:** Ensuring that all data transmitted between users and the system is encrypted end-to-end can prevent interception and unauthorized access. This enhancement includes encrypting data at rest and in transit using robust encryption algorithms.
- **Database Encryption:** Encrypting sensitive data stored in databases adds an extra layer of protection against data breaches. This measure ensures that even if the database is compromised, the data remains unreadable without the decryption keys.

5. Secure Development Lifecycle (SDLC):

- **Integration of Security in SDLC:** Embedding security practices throughout the software development lifecycle ensures that security is considered at every stage, from design to deployment. This includes threat modeling, secure coding practices, and regular security testing.
- **Developer Training:** Providing ongoing security training for developers to keep them informed about the latest security threats and best practices. This training helps developers write more secure code and identify potential vulnerabilities during development.

6. Enhanced Access Control:

- **Role-Based Access Control (RBAC):** Implementing RBAC can ensure that users have the minimum necessary access to perform their tasks. This approach reduces the risk of unauthorized access and limits the potential impact of compromised accounts.
- **Audit Logging:** Maintaining comprehensive audit logs of user activities can help track access and detect suspicious behavior. These logs are essential for forensic analysis and incident response.

By implementing these future security enhancements, the Foodereum project can achieve a higher level of security, ensuring that it remains resilient against evolving threats and provides a secure environment for users and their data. These enhancements will build on the existing security measures, creating a robust and comprehensive security framework.

BIBLIOGRAPHY

- [1] J. Hamlett, “Tips to Overcome Supply Chain Challenges for Food Manufacturers — food-safety.com.” <https://www.food-safety.com/articles/8792-tips-to-overcome-supply-chain-challenges-for-food-manufacturers>, 2023.
- [2] J. Asaad, “Fixing the 5 Big Problems in the Food Supply Chain — supplychainbeyond.com.” <https://supplychainbeyond.com/5-big-problems-in-the-food-supply-chain/>, 2022.
- [3] Klarys.io, “Major challenges in the fresh food supply chain — klarys.io.” <https://www.klarys.io/en/articles/fresh-food-6-major-supply-chain-challenges>, 2023.
- [4] J. Payne, “Overcoming the 4 Biggest Food Supply Chain Issues — aptean.com.” <https://www.aptean.com/en-US/insights/blog/biggest-food-supply-chain-issues>, 2023.
- [5] L. Lacerta Group, “Food Supply Chain: Trends and Challenges for 2024 — blog.lacerta.com.” <https://blog.lacerta.com/blog/food-supply-chain>, 2024.
- [6] M. O. Grida, S. Abd Elrahman, and K. A. Eldrandaly, “Critical success factors evaluation for blockchain’s adoption and implementing,” *Systems*, vol. 11, no. 1, 2023.
- [7] UNDP, “Blockchain for Agri-Food Traceability — undp.org.” <https://www.undp.org/publications/blockchain-agri-food-traceability>, 2021.
- [8] R. M. Ellahi, L. C. Wood, and A. E.-D. A. Bekhit, “Blockchain-based frameworks for food traceability: A systematic review,” *Foods*, vol. 12, no. 16, 2023.
- [9] S. (n.d.), “Blockchain for the Food Supply Chain: All You Need to Know — scnsoft.com.” <https://www.scnsoft.com/blockchain/food-supply-chain>.
- [10] B. W. Sam Daley, “Blockchain for Food: 10 Examples to Know — builtin.com.” <https://builtin.com/blockchain/food-safety-supply-chain>, 2023.
- [11] A. Sristy, “Blockchain in the food supply chain - What does the future look like? — tech.walmart.com.” https://tech.walmart.com/content/walmart-global-tech/en_us/news/articles/blockchain-in-the-food-supply-chain.html, 2021.

- [12] @nhsz, “Introduction to dapps — ethereum.org — ethereum.org.” <https://ethereum.org/en/developers/docs/dapps/>, 2024.
- [13] QuickNode, “How to Connect Your Dapp With MetaMask Using Ethers.js — QuickNode — quicknode.com.” <https://www.quicknode.com/guides/ethereum-development/dapps/how-to-connect-your-dapp-with-metamask-using-ethersjs>, 2023.
- [14] M. (n.d.), “Home — MetaMask developer documentation — docs.metamask.io.” <https://docs.metamask.io>.
- [15] O. SINGH, “How to use Ganache for blockchain project development — cointelegraph.com.” <https://cointelegraph.com/news/how-to-use-ganache-for-blockchain-project-development>, 2023.
- [16] R. IDE, “Solidity Compiler & Remix - Ethereum IDE 1 documentation — remix-ide.readthedocs.io.” <https://remix-ide.readthedocs.io/en/latest/compile.html>.
- [17] M. P. Caro, M. S. Ali, M. Vecchio, and R. Giaffreda, “Blockchain-based traceability in agri-food supply chain management: A practical implementation,” *2018 IoT Vertical and Topical Summit on Agriculture - Tuscany (IOT Tuscany)*, pp. 1–4, 2018.
- [18] A. Marchese and O. Tomarchio, “A Blockchain-Based System for Agri-Food Supply Chain Traceability Management,” *SN Computer Science*, vol. 3, p. 279, May 2022.
- [19] Shivendra, K. Chiranjeevi, M. K. Tripathi, and D. D. Maktedar, “Block chain technology in agriculture product supply chain,” in *2021 International Conference on Artificial Intelligence and Smart Systems (ICAIS)*, pp. 1325–1329, 2021.
- [20] H. Xiong, T. Dalhaus, P. Wang, and J. Huang, “Blockchain Technology for Agriculture: Applications and Rationale,” *Frontiers in Blockchain*, vol. 3, 2020.
- [21] H. Xiong, T. Dalhaus, P. Wang, and J. Huang, “Blockchain technology for agriculture: Applications and rationale,” *Frontiers in Blockchain*, vol. 3, 2020.
- [22] A. Shahid, U. Sarfraz, M. W. Malik, M. S. Iftikhar, A. Jamal, and N. Javaid, “Blockchain-based reputation system in agri-food supply chain,” in *International Conference on Advanced Information Networking and Applications*, 2020.
- [23] H. R. Hasan and K. Salah, “Blockchain-based proof of delivery of physical assets with single and multiple transporters,” *IEEE Access*, vol. 6, pp. 46781–46793, 2018.

- [24] S. S. Sahoo and V. K. Chaurasiya, “Proof of location based delivery system using multi-party virtual state channel: a blockchain model,” *The Journal of Supercomputing*, vol. 80, pp. 703–733, Jan. 2024.
- [25] A. Alketbi, Q. Nasir, and M. A. Talib, “Blockchain for government services — use cases, security benefits and challenges,” in *2018 15th Learning and Technology Conference (L&T)*, pp. 112–119, 2018.
- [26] J. Clavin, S. Duan, H. Zhang, V. P. Janeja, K. P. Joshi, Y. Yesha, L. C. Erickson, and J. D. Li, “Blockchains for government: Use cases and challenges,” *Digit. Gov.: Res. Pract.*, vol. 1, nov 2020.
- [27] V. Kumaresh, G. Kumar, R. K. Bhat, and N. J, “Foodereum: A blockchain-based authenticated solution for food supply chain,” in *2022 IEEE North Karnataka Subsection Flagship International Conference (NKCon)*, pp. 1–5, 2022.

Appendix A

SOLIDITY SMART CONTRACT CODE

```
// Foodereum.sol
pragma solidity ^0.6.0;

contract Foodereum {
    event Added(uint256 index);

    struct State {
        string details;
        address user;
    }

    struct FoodProduct {
        address creator;
        string productName;
        uint256 productId;
        string date;
        uint256 totalStates;
        mapping (uint256 => State) positions;
    }

    mapping(uint => FoodProduct) allProducts;
    uint256 items=0;
```

```
function concat(string memory a,string memory b) public pure returns (string
memory) {
    return string(abi.encodePacked(a,'',b));
}
```

```
function sanction(string memory _text, string memory _date) public returns
(bool) {
    FoodProduct memory sanction = FoodProduct({creator: msg.sender,
        totalStates: 0,productName: _text, productId: items, date: _date});
    allProducts[items]=sanction;
    items = items+1;
    emit Added(items-1);
    return true;
}
```

```
function update(uint _productId, string memory info) public returns (string
memory) {
    require(_productId<=items);
    State memory newState = State({user: msg.sender, details: info});
    allProducts[_productId].positions[ allProducts[_productId].totalStates
        ]=newState;
    allProducts[_productId].totalStates = allProducts[_productId].totalStates
        +1;
    return info;
}
```

```
function trackProduct(uint _productId) public returns (string memory) {
    require(_productId<=items);
    string memory output="<b>Product Name: </b>";
```

```
output=concat(output, allProducts[_productId].productName);
output=concat(output, "<br><b>Sanctioned Date: </b>");
output=concat(output, allProducts[_productId].date);

for (uint256 j=0; j<allProducts[_productId].totalStates; j++){
    output=concat(output, allProducts[_productId].positions[j].details);
}
return output;
}
}
```

Appendix B

SCREENSHOTS

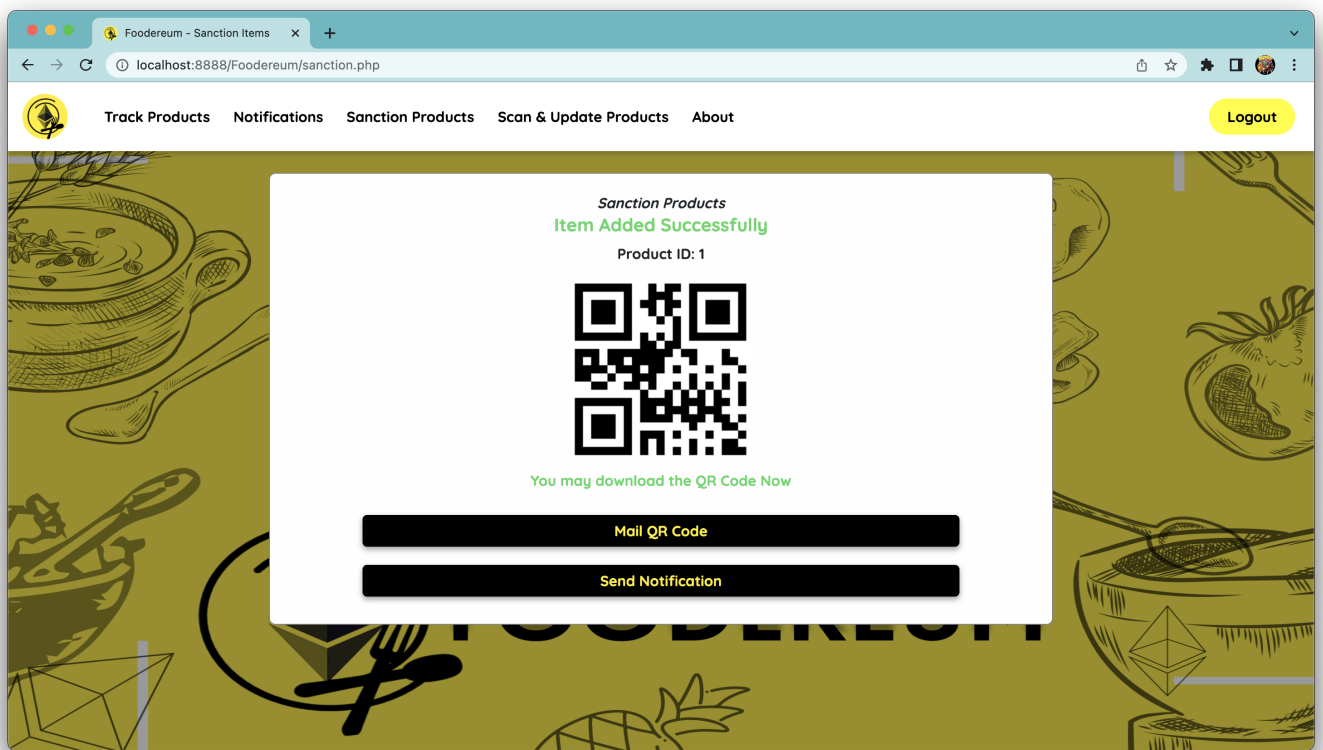


Figure B.1: Sanction Products Page

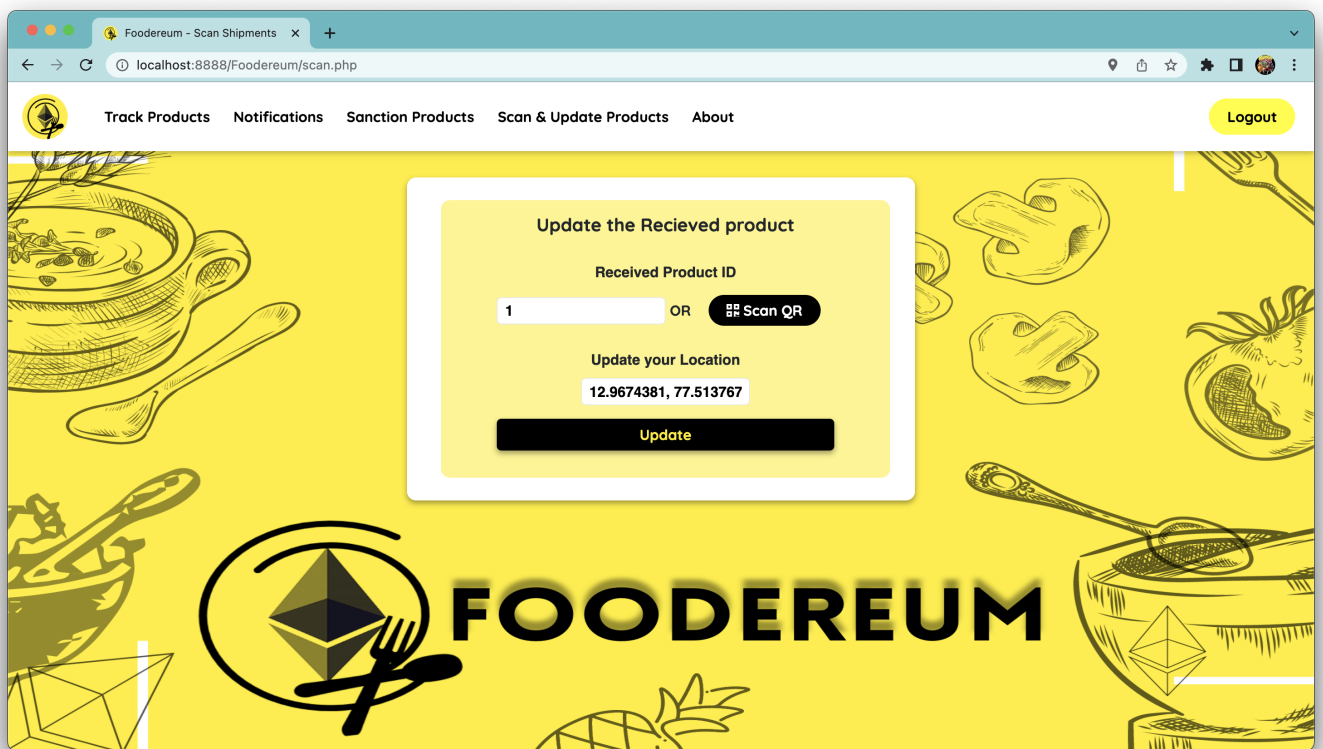


Figure B.2: Scan & Update Products Page

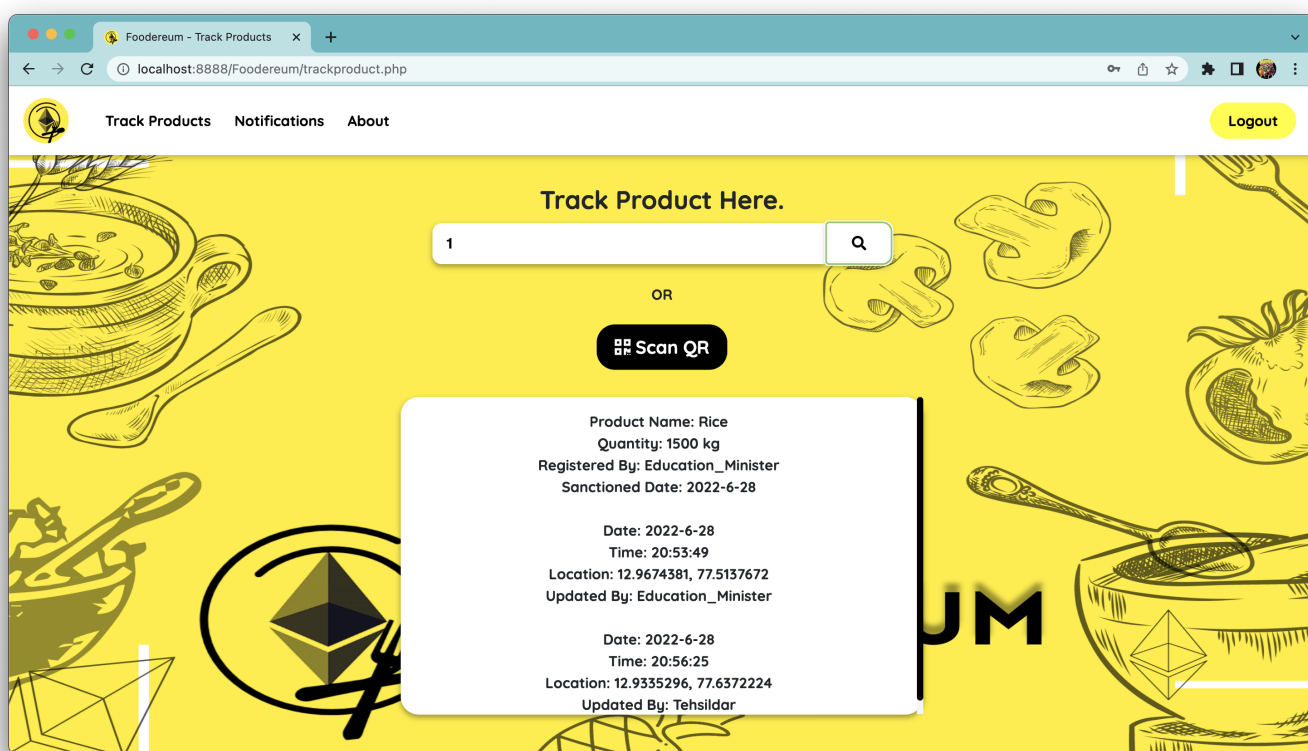


Figure B.3: Track Products Page

Foodereum | Notifications

localhost:8888/Foodereum/manage.php

Title

Message

Broadcast Time: Now

For: Education_Minister

Send Message

Recorded Notifications:

No	Sender	Title	Message	Receiver	Timestamp
1	Tehsildar	Rice 2000kg	Not	Education_Minister	2022-05-14 12:34:27
2	Education_Minister	Rice and Oil Sent	Rice 1500kg and Oil 500ml Sent	Institute	2022-06-28 21:07:01

Figure B.4: List of Notifications